Looped Transformers and Recurrent Depth: The Full Report

The Loop technology behind GPT-6 Astra · five core papers in depth and trend insights (2018–2026)

awesome_loop_transformer · https://github.com/asimfish/awesome_loop_transformer

2026-09-06

Contents

Abstract 1

How to read this report 1

1. Why now 1

2. Concepts and taxonomy 2
- 2.1 What a looped transformer is . . . 2
- 2.2 Four loop granularities . . . 2
- 2.3 Three orthogonal axes . . . 2

3. An eight-year timeline 3

4. The five core papers at a glance 3
- 4.1 How the five fit together . . . 4

5. The wider literature map 4
- 5.1 Theory: expressivity, ICL, length generalization . . . 4
- 5.2 Scaled pretraining . . . 4
- 5.3 Efficiency: latency, KV memory, quantization . . . 4
- 5.4 Stability and training theory . . . 5
- 5.5 Adaptive computation and halting . . . 5
- 5.6 Retrofit and training-free . . . 5
- 5.7 Post-training and reinforcement learning . . . 5
- 5.8 Interpretability and dynamics . . . 6
- 5.9 Tiny recursive reasoners . . . 6
- 5.10 Cross-modal and applications . . . 6

6. Ten trend insights 6
- Insight 1: Recurrent depth passed the compute-matching gate — through a narrow door . . . 6
- Insight 2: “How many loops” splits into two regimes . . . 7
- Insight 3: Loop granularity is getting finer and converging on the middle . . . 7
- Insight 4: ODE flow or fixed-point iteration? Two philosophies are diverging . . . 7
- Insight 5: Training stability went from folklore to theorems . . . 8
- Insight 6: Inference cost is the last big bottleneck, now decomposed into three sub-problems . . . 8
- Insight 7: Latent looping and explicit CoT are complements, not substitutes . . . 8
- Insight 8: The capability profile is now clear — loops manipulate knowledge, they do not store it . . . 8
- Insight 9: Applications are expanding from math puzzles to agents and embodiment . . . 8
- Insight 10: Productization has started; the toolchain has not caught up . . . 8

7. Inferences about GPT-6 Astra (explicitly labeled as inference) 9

8. Open problems 9

9. Conclusion 9

Appendix A: Circulation errors corrected in this report 9

Appendix B: Method and sources 10

Appendix C: Glossary 10

Report 01 · Universal Transformers: Where Recurrent Depth Began 11
- At a glance . . . 11
- One sentence . . . 11
- 1. The problem . . . 11
- 2. Method . . . 12
- 3. Key experiments and numbers . . . 12
- 4. Why it matters . . . 13
- 5. Limitations . . . 13
- 6. Relation to other work . . . 13
- 7. Practical takeaways . . . 13
- Appendix: quick numbers . . . 13
- Glossary . . . 13

Report 02 · Scaling up Test-Time Compute with Latent Reasoning: Recurrent Depth at 3.5B 15
- At a glance . . . 15
- One sentence . . . 15
- 1. The problem . . . 15
- 2. Method . . . 16
- 3. Key experiments and numbers . . . 16
- 4. Two failed runs: why scaling is hard (§4.3) . . . 17
- 5. Capabilities that come for free (§6) . . . 18
- 6. What happens in latent space (§7) . . . 18
- 7. Limitations . . . 18
- 8. Relation to other work . . . 18
- 9. Practical takeaways . . . 18
- Appendix: quick numbers . . . 19
- Glossary . . . 19

Report 03 · Training-Free Looped Transformers: A Loop Wrapper for Frozen Checkpoints 20
- At a glance . . . 20
- One sentence . . . 20
- 1. The problem . . . 20
- 2. Method . . . 21
- 3. Key experiments and numbers . . . 22
- 4. Why it matters . . . 23
- 5. Limitations . . . 23
- 6. Relation to other work . . . 23
- 7. Practical takeaways . . . 23
- Appendix: quick numbers . . . 23
- Glossary . . . 23

Report 04 · LoopCoder-v2: In a Parallel Loop Transformer, the Second Loop Is Almost Everything 25
- At a glance . . . 25
- One sentence . . . 25
- 2. Background: why PLT . . . 25
- 3. Question and method . . . 26
- 4. Key results . . . 26

5. Where the conclusion applies (important) . . . 27
6. Limitations . . . 28
7. Relation to other work . . . 28
8. Practical takeaways . . . 28
Appendix: quick numbers . . . 28
Glossary . . . 28

Report 05 · Loop the Loopies!: The First Compute-Matched Win for Looped MoE **30**
At a glance . . . 30
One sentence . . . 30
1. The problem . . . 30
2. Method . . . 31
3. Key results . . . 32
4. Why it matters . . . 33
5. Limitations and caveats . . . 33
6. Relation to other work . . . 33
7. Practical takeaways . . . 33
Appendix: quick numbers . . . 34
Glossary . . . 34

Abstract

Reusing one set of parameters along the depth axis — an idea introduced by the Universal Transformer in 2018 — became one of the fastest-growing directions in large-model architecture research in 2025–2026, and reached public attention in September 2026 when The Information reported that GPT-6 Astra uses "recurrent depth". This report takes the five papers threaded together by the WeChat article "The Loop technology behind GPT-6 Astra" as its skeleton, re-reads each against the original text to restore the preconditions and numbers that secondary coverage dropped, and then uses an exhaustive arXiv search to place roughly 60 directly related works from 2018–2026 into ten themes and ten trend judgments. Three conclusions stand out. First, recurrent depth has for the first time beaten a vanilla MoE transformer under **measured compute matching** (Loopie, July 2026), but the win sits in a narrow configuration: two steps, per-layer looping, MoE backbone. Second, "how many loops" splits into two regimes — under a fixed pretraining budget the answer is 2; with variable inference compute the answer is "more, adaptively" — and LoopCoder-v2's "three or more regress" is the price of a parallel-loop design, not a universal ceiling on recurrent depth. Third, latent looping and explicit chain-of-thought are complementary axes of test-time compute, and stacking them yields more than the sum of their parts.

How to read this report

Three reading paths, by available time:

Time	Read	You get
5 minutes	Abstract → the headings and bold sentences of §6 → Appendix A	Where the field stands and which popular claims are inaccurate
30 minutes	§2–4 (concepts, timeline, five-paper table) → all of §6 → §8 open problems	A complete map; you can place a new paper in its cell
2 hours	Everything + the five reports in `reports/en/01–05`	Every key number traceable to a table in the original paper

Unfamiliar terms are defined in **Appendix C**. Diagram labels are in Chinese; each caption carries the full meaning. Everything about GPT-6 Astra is labeled as inference.

1. Why now

On 2 September 2026 The Information reported that OpenAI's newly released GPT-6 Astra uses an architectural technique called recurrent depth. Researcher @nrehiew_ then traced the lineage on X through five papers, and the WeChat account PaperAgent turned that thread into a Chinese long-form article. This report's direct goal is to read those five papers closely

and put them back into the larger literature.

On Astra itself this report does exactly one thing: separate what can be verified from what cannot. **Verifiable**: the contents, numbers, and relations of the five papers. **Not verifiable**: which kind of looping Astra uses, how many loops, how it was trained — OpenAI's release page discloses no architectural detail, The Information's report rests on anonymous sources, and neither primary document was accessible to us. Every statement about Astra below is labeled as inference.

2. Concepts and taxonomy

2.1 What a looped transformer is

In a standard transformer forward pass each layer's parameters are used once; depth equals parameter count divided by per-layer parameters. A **looped transformer** (also looped / recurrent-depth / weight-tied / universal transformer) applies the same parameters repeatedly, decoupling depth from parameter count: a 4-layer block looped 32 times has an effective depth of 128 layers.

2.2 Four loop granularities

Granularity	Definition	Representatives
Model-loop	The whole stack's output is fed back as input	Universal Transformer, Huginn's core, Ouro
Block-loop	Only a contiguous middle segment loops; first and last layers do not	Huginn's prelude–core–coda, Hyperloop, Training-Free Looped, SMELT
Layer-loop	Each layer repeats K times before passing to the next	Loopie, Training-Free layer-mode, MixerLoop (mixer only)
Latent conditioning	The previous loop's (or previous token's) hidden state is injected as a condition	Coconut, PLT's CLP, Latent Recurrent Transformer



Figure 1 (labels in Chinese): model-loop repeats the whole stack; block-loop repeats only the middle segment with input injection; layer-loop repeats each layer before the next. Effective depth = stored layers × loops, parameters unchanged.

These are not exclusive. Loopie is layer-loop; Huginn is block-loop at the three-stage level and model-loop inside its core; PLT is model-loop plus latent conditioning (the previous round's state shifted by one position).

2.3 Three orthogonal axes

Beyond granularity, three independent design axes:

1. **Fixed versus adaptive loop count.** Fixed (every token gets the same compute: Loopie, LoopCoder-v2); random at training, free at inference (Huginn); per-token adaptive halting (UT's ACT, Mixture-of-Recursions, Think-at-Hard, RecurTrace).
2. **From scratch, retrofit, or training-free.** Pretrain from scratch (Huginn, Ouro, Loopie, Nanbeige4.2); convert an existing model and continue training (Relaxed Recursive Transformers, Retrofitted Recurrence); no training at all, wrap

at inference (Training-Free Looped Transformers, Program-of-Layers).

3. **What the objective treats the loop as.** Plain language modeling with truncated unrolling (Huginn); fixed point / equilibrium (DEQ, Attractor Models, Fixed-Point Reasoners, STARS); refinement of an ODE integration (Training-Free); latent CoT with intermediate supervision (LOTUS, LoopMTP); RL with rewards on loop steps (RLTT, LoopRPT).

3. An eight-year timeline

When	Event
2016	Neural GPU (Kaiser & Sutskever), Adaptive Computation Time (Graves): precursors of looping + adaptive halting
2018-07	**Universal Transformers** (Dehghani et al.): parameters shared across time steps, coordinate embeddings, ACT; ICLR 2019
2019	ALBERT uses cross-layer sharing for compression; Deep Equilibrium Models replace explicit unrolling with fixed-point solving
2021–2022	Deep-thinking line (Schwarzschild, Bansal): random unrolling + input injection lets recurrent nets extrapolate easy→hard; path-independent equilibrium models (Anil)
2023	Looped Transformers as Programmable Computers (Giannou): a 13-layer looped transformer emulates a general computer; Looped Transformers are Better at Learning Learning Algorithms (Yang): <10% parameters match standard ICL; Sparse Universal Transformer
2024	MoEUT: first shared-layer model to slightly beat standard transformers on LM; Relaxed Recursive Transformers (per-layer LoRA); Looped Transformers for Length Generalization; Coconut reasons in continuous space
2025-02	**Scaling up Test-Time Compute with Latent Reasoning** (Geiping et al.): Huginn-3.5B, 800B tokens, coins "recurrent depth"; Reasoning with Latent Thoughts (Saunshi) proves k layers looped L times ≈ kL layers
2025-05	To CoT or To Loop (Xu & Sato): loops excel at DAG evaluation, CoT at self-reducible problems
2025-06–10	HRM (27M parameters solves Sudoku/ARC), Mixture-of-Recursions, TRM, Ouro (2.6B, 7.7T tokens), Parallel Loop Transformer, parallel samplers (looped models ≈ diffusion LMs)
2025-11	Retrofitted Recurrence (McLeish); Think-at-Hard (loop only on hard tokens)
2026-01–04	LoopFormer (ICLR), Depth-Recurrent Attention Mixtures, RLTT (ICML), LoopRPT, Adaptive Loops and Memory, Hyperloop Transformers, iso-depth scaling law (φ=0.46), Loop-Think-Generalize (COLM)
2026-05	**Training-Free Looped Transformers** (UT Austin); MELT, LT2, LoopQ, STARS (ICML), Fully Looped Transformer, Attractor Models, Dual-Path
2026-06	**LoopCoder-v2** (7B PLT, 18T tokens, "only loop once"); Fixed-Point Reasoners; 1/N residual scaling theory; LOTUS closes the latent-vs-explicit CoT gap; Program-of-Layers (ICML)
2026-07	**Loop the Loopies!** (Loopie-20B-A2B, first compute-matched win); DeepLoop; Looped Latent Attention (32× KV compression); Nanbeige4.2-3B (28T-token looped model shipped as an agentic product); Hidden Decoding (WeChat's WeLM 617B takes the sequence-length route)
2026-08	MixerLoop (loop the mixer only), Gated Recurrent Transformers, Looped LMs improve compositional tool calling, Think Shallow Solve Deep (settling/drifting taxonomy)
2026-09	**SMELT**: Chinchilla-style scaling laws for compute-matched looped MoE, 6.8–18% training-FLOP savings; RecurTrace; GPT-6 Astra released, The Information reports recurrent depth

4. The five core papers at a glance

Full reports are in `reports/en/01–05`. A comparison table:

	Universal Transformers	Recurrent Depth (Huginn)	Training-Free Looped	LoopCoder-v2	Loop the Loopies
Date	2018-07	2025-02	2026-05	2026-06	2026-07
Group	UvA / DeepMind / Google	ELLIS / UMD / LLNL	UT Austin	Beihang / IQuest	IQuest
Granularity	model-loop	block (three-stage)	middle 4 layers, block or layer	model-loop (PLT)	layer-loop
Training	from scratch	from scratch, 800B tokens	**none**	from scratch, 18T tokens	from scratch, 3.5T tokens
Loop count	ACT adaptive	random $\bar{r}$=32 at train, free at test	K=3-stage RK	R=2 (train = test)	R=2
Scale	~65M	3.5B dense	0.6B–30B (existing models)	7B dense	20B-A2B / 6B-A0.6B MoE
Headline number	bAbI error 22.1→0.29	r 4→32: ARC-E 49.1→69.9	45 cells 87% non-negative, MMLU-Pro +2.64	SWE-bench Verified 43.0→64.4→27.6	overtakes Qwen3-30B-A3B after 600B tokens at matched compute

	Universal Transformers	Recurrent Depth (Huginn)	Training-Free Looped	LoopCoder-v2	Loop the Loopies
Core contribution	defines recurrent depth + halting	first modern-scale training; latent orbits	layer = Euler step; sub-step, don't repeat	mechanistic diagnosis of two-loop saturation; CoT + loop super-additive	Loopie Recipe; SPT
Most misread point	–	the "same 0.8T" baseline is really 0.18T	gains cap at ~2–3 points	non-monotonicity is PLT-specific	"compute-matched" = measured step time, not FLOPs

4.1 How the five fit together

The five papers form one argument:

1. UT proposes the mechanism; parameter–compute ratio and training stability remain open.
2. Huginn answers "can it be trained at modern scale" — yes, at the cost of inference compute and knowledge capacity.
3. Training-Free answers "can existing models get it for free" — partly — and supplies the ODE explanation and the layer-mode requirement for MoE.
4. LoopCoder-v2 answers "how many loops at deployment" — flattens latency with PLT, finds saturation at R=2, and diagnoses why.
5. Loopie answers "can it win at matched compute" — yes, via layer-loop + MoE + measured step-time matching, also at R=2.

5. The wider literature map

Categories below come from arXiv searches (looped/recurrent-depth/depth-recurrent transformer; latent reasoning □ loop/recurrence/recursive; named models), keeping work directly relevant to LM architecture. The full list is in the README and `data/papers.csv`.

5.1 Theory: expressivity, ICL, length generalization

- Giannou et al. 2023 (ICML): looped transformers as programmable computers.
- Yang et al. 2024 (ICLR): looped models learn learning algorithms with <10% of the parameters.
- Gatmiry et al. 2024 (ICML): can looped transformers implement multi-step GD; Chen et al. 2025 (AISTATS): bypassing the exponential dependency.
- Fan et al. 2025 (ICLR): adaptive-step looped transformers substantially improve length generalization.
- Saunshi et al. 2025 (ICLR): many reasoning problems need depth, not parameters; k layers looped L times ≈ kL layers; looped models simulate T steps of CoT.
- Xu & Sato 2025: formal separation — loops for deterministic parallel computation (DAG evaluation), CoT for approximate inference over self-reducible problems.
- Gong et al. 2025: loss-landscape view — looping induces a River-V-Valley bias.
- Zhang et al. 2026: the "budget law": weight-tied loops learn algorithms with speed v ≈ n_train/T_train; halting rule T* = □n/v̂□.
- Wu et al. 2026: looped transformers with LayerNorm provably learn the power method.

5.2 Scaled pretraining

Model	Size	Tokens	Loops	Notes
Huginn-0125	3.5B dense	0.8T	r̄=32, free at test	first modern scale; open
Ouro 1.4B / 2.6B	dense	7.7T	4 + entropy-regularized depth allocation	matches 12B SOTA; gains from knowledge manipulation, not storage; open
LoopCoder-v2	7B dense PLT	18T	2	coder; open
Loopie-20B-A2B / 6B-A0.6B	MoE	3.5T	2 (layer-loop)	first compute-matched win; open
Nanbeige4.2-3B	3B	28T	2 (stack reuse)	agentic product model, beats Qwen3.5-9B and Gemma4-12B
SMELT	MoE to 54B non-embedding	–	middle half ×2	scaling law: saves 6.8–18% training FLOPs

Common to scaled pretraining: **loop counts are small (2–4)** except Huginn; **the three newest are layer-loop or mid-stack**; **MoE backbones are the default**.

5.3 Efficiency: latency, KV memory, quantization

- **Latency**: Parallel Loop Transformer (cross-loop position offset, latency ≈ one forward); Efficient Parallel Samplers (Geiping 2025) treat looped models as diffusion LMs, a diffusion-forcing sampler gives 5× speedup with no tuning; Continuous Depth-wise Batching (Bae 2025).
- **KV memory**: Huginn's looped KV sharing (budget 4, no loss); Mixture-of-Recursions' KV-sharing variant; MELT (one shared KV per layer with gated updates, constant memory after fine-tuning Ouro); **Looped Latent Attention** (cross-loop KV trajectories are low-rank; SVD codec near-lossless at 32×; Ouro-1.4B batch capacity on one H200 from 32 to 768); CHASE (skip-exit for looped SSMs).
- **Attention complexity**: LT2 (linear/sparse attention + loops; looping lets linear attention refine memory iteratively and sparse attention expand its receptive field; Ouro-hybrid-1.4B converted with ~1B tokens).
- **Quantization**: LoopQ (first systematic study; under W4A4, +68.8% average accuracy over the strongest PTQ baseline).
- **Deployment notes**: Halloran 2026 fixes five bugs in Nanbeige4.2-3B on Apple Silicon, notes that layer reuse doubles peak attention memory, and extends usable context 2.7× with chunked prefill.

5.4 Stability and training theory

- Huginn's two failed runs (hidden-state collapse, recurrence collapse) and sandwich normalization.
- Wang et al. 2026 (residual scaling): weight sharing correlates residual updates across iterations, requiring ε=1/N rather than 1/√L; factored ε=λ/(N√L); optimal LR depends only on the number of unique layers L, so it transfers from small to large N.
- DeepLoop: the Post-LN DeepNorm exponent must rise from 1/4 to 1/2; $\alpha = (2N)^{1/2}$, $\beta = (8N)^{-1/2}$.
- Fully Looped Transformer: distribute inter-loop signals to all layers + attention injection; stable to 12 loops.
- STARS (ICML 2026): Jacobian spectral-radius regularization + random loop sampling drives latent states to asymptotically stable fixed points, mitigating "peak then collapse".
- Fixed-Point Reasoners: pre-norm + residual scaling + fixed-point convergence as halting.
- Think Shallow, Solve Deep: an operator's finite-time regime (settling / marginal / drifting) predicts whether extra depth is safe; one terminal fixed-point objective moves regime and depth behavior together; Huginn is non-settling.
- Stabilizing Extrapolation via Learned Stochastic Stopping: randomizing loop count in training sharply reduces OOD variance.

5.5 Adaptive computation and halting

- Classics: ACT (2016), PonderNet (2021), CALM (2022).
- Mixture-of-Recursions (NeurIPS 2025): lightweight routers assign per-token recursion depth; attention and KV only for active tokens; a new Pareto frontier at 135M–1.7B.
- Think-at-Hard (ICML 2026): "latent overthinking" — most tokens are right after one pass and later iterations sometimes break them; loop only on the hard 7% of tokens, +3.8–4.4%.
- LoopFormer (ICLR 2026): shortcut-consistency training aligns trajectories of different lengths for budget-elastic inference.
- RecurTrace (Sep 2026): loop-memory attention (attend to one's own earlier loop states) + an oracle-supervised halting head; 56.9% on MathQA at 2.0 loops on average, while ACT/PonderNet collapse to one loop.
- Adaptive Depth in Looped Transformers (Popescu): diagnosing learned halting gates.
- Two-Scale Latent Dynamics: an early-exit rule on the second-order step-size difference beats Huginn's KL criterion.

5.6 Retrofit and training-free

- Relaxed Recursive Transformers (ICLR 2025): convert Gemma 2B and others into a shared block + per-layer LoRA, recovering most performance.
- Retrofitted Recurrence (McLeish 2025): a curriculum that gradually raises loop count; beats post-training the original on math at equal compute.
- Training-Free Looped Transformers (2026): zero training, RK sub-steps on the middle 4 layers.
- Program-of-Layers (ICML 2026): a small network emits a per-input program of which layers to skip or repeat, often more accurate with fewer layers.
- Retrofitting Recurrent Depth (Shapiro 2026): installation, extrapolation, transfer, and retention at two parameter budgets.

5.7 Post-training and reinforcement learning

- LoopCoder-v2: a Thinking mode (explicit CoT) on top of the R=2 loop, LiveCodeBench +26.9, super-additive.
- Loopie: SPT (SFT masking + PT scale, 2T tokens over 10 epochs without overfitting or forgetting) + GSPO/DAPO RL.
- RLTT (ICML 2026): standard GRPO credits only the final latent state, mismatching the loop's internal computation; distributing reward along the latent trajectory gives Ouro-2.6B +10.9% on math.
- LoopRPT: reframes next-token prediction as reasoning and assigns RL signal directly to latent steps via an EMA teacher, compressing effective reasoning into fewer iterations.
- LOTUS (Fan 2026): K latent blocks processed in parallel with a cross-entropy loss on each position's gold CoT token; first latent-CoT method to match explicit CoT at 3B, cutting thought-phase latency 2.5–6.9×.
- LoopMTP: softly aligns loop-t hidden state with the token t steps ahead; stable to 15 loops.
- Dense Supervision Is Not Enough: the "readout blind spot" of looped LMs.

5.8 Interpretability and dynamics

- Huginn paper: orbits, sliders, fixed points; path independence.
- Lu et al. 2025: logit-lens probes of Huginn find limited evidence of interpretable latent CoT; probe results depend heavily on layer and decoding method.
- Two-Scale Latent Dynamics: small within-block refinements plus larger between-block drifts; loop steps shrink and become orthogonal over training.
- Jacobian Lens (Sep 2026): Ouro rebuilds a "global workspace" every loop and linear transport cannot cross loop boundaries; Huginn carries content across 16 recurrences but reads/writes act within a ~2-recurrence window.
- LoopCoder-v2: effective rank peaks at loop 2 then declines; update directions oscillate.
- Observable Patterns Are Not Explanations: a causal–geometric analysis warning against reading observable patterns as explanations.
- Step-resolved data attribution for looped transformers.

5.9 Tiny recursive reasoners

A parallel branch that does not do language modeling but solves hard structured puzzles with millions of parameters:

- HRM (Jun 2025): 27M parameters, 1000 samples, two recurrent modules at different frequencies; near-perfect Sudoku/Maze, beats larger models on ARC.
- TRM (Oct 2025): a single 2-layer 7M network; ARC-AGI-1 45%, ARC-AGI-2 8%, above most LLMs.
- Attractor Models (May 2026): fixed-point implicit differentiation, training memory independent of effective depth; 27M parameters 91.4% on Sudoku-Extreme; a 770M model beats a 1.3B transformer trained on 2× tokens; "equilibrium internalization" lets the solver be removed at inference.
- Denoising Recursion Models: multi-step denoising recursion; beats TRM on ARC-AGI.
- Critical analyses: Hierarchical Reasoning Models: Perspectives and Misconceptions; Are Your Reasoning Models Reasoning or Guessing.

5.10 Cross-modal and applications

- Vision: LoopViT (18M parameters, 65.8% on ARC-AGI-1, beating 73M ensembles), RD-ViT (semantic segmentation), ELT (visual generation), Déjà View (multi-view 3D reconstruction).
- Robotics: Recurrent-Depth VLA (tasks at 0% success with one iteration exceed 90% with four; 80× faster than token-reasoning VLAs).
- Speech: Depth-Conditioned Looped Transformers for ASR.
- Agents: Looped LMs Improve Compositional Tool Calling (multi-step tool use rises with recurrent depth); Nanbeige4.2-3B's agentic deployment.
- Multimodal LMs: Looping Back to Move Forward (recursive LMMs).

6. Ten trend insights

Insight 1: Recurrent depth passed the compute-matching gate — through a narrow door

Loopie (Jul 2026) and SMELT (Sep 2026) are the first wins against vanilla models at **matched training cost**: Loopie matches measured step time and overtakes a reproduced Qwen3-30B-A3B after 800B tokens; SMELT matches FLOPs, parameters, and KV simultaneously, fits scaling laws, saves 6.8–18% of training FLOPs, and the advantage does not vanish

with scale (to 54B non-embedding parameters). Together they move looping from a parameter-efficiency trick to a compute-efficiency one.

The winning configuration is narrow: **MoE backbone, layer-loop or mid-stack looping, two loops.** Schwethelm et al.'s iso-depth scaling law explains why: one recurrence is worth $r^{0.46}$ unique parameters (0.38 under truncated backprop), so substituting loops for parameters loses under FLOP matching. Loopie's win does not trade parameters for loops; it converts the **activation memory** saved by looping into larger micro-batches and then into width — an infrastructure dividend invisible in theoretical FLOPs.

Insight 2: "How many loops" splits into two regimes

The reported optima side by side: Loopie 2, LoopCoder-v2 2, SMELT 2, Nanbeige4.2 2, Ouro 4, Huginn 32 (up to 64), RecurTrace 2.0 on average, Think-at-Hard 1 on 93% of tokens.

The dividing line is **where the compute constraint binds**. Under a fixed pretraining budget, each extra loop multiplies training FLOPs and the marginal return decays fastest after R=2 (Loopie Figure 6): the answer is 2. With variable inference compute, one model can loop more on hard inputs and less on easy ones (Huginn's ARC-C saturates at 8 loops zero-shot and 32 loops with 25–50 shots): the answer is "as needed, adaptively".

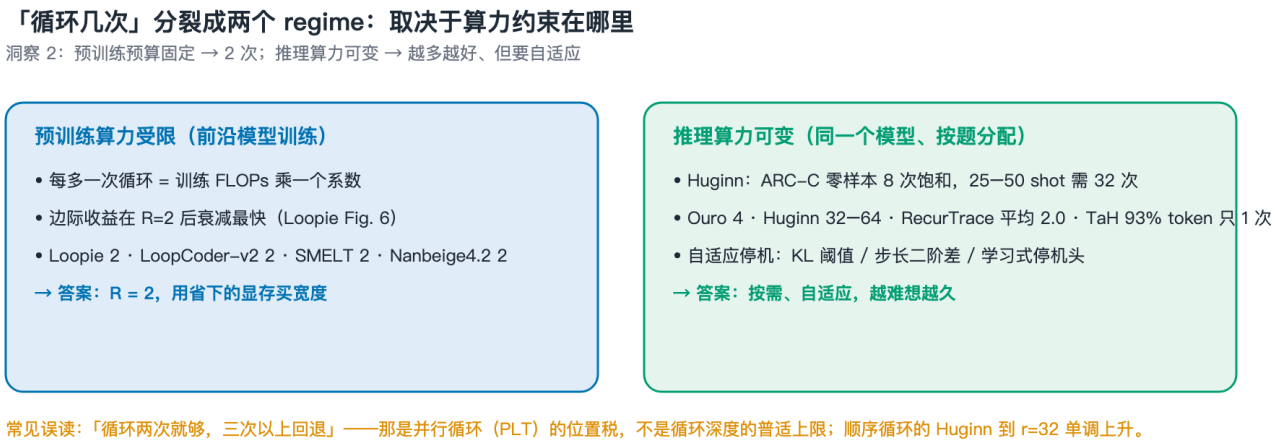


Figure 2 (labels in Chinese): left, fixed pretraining budget → two loops (Loopie, LoopCoder-v2, SMELT, Nanbeige); right, variable inference compute → adaptive and deep (Huginn 8 → 32 loops with more context; Ouro 4; RecurTrace 2.0 avg; Think-at-Hard loops only 7% of tokens).

LoopCoder-v2's "three or more regress" is often quoted as a universal ceiling. The paper's own explanation is that PLT's cross-loop position offset charges a fixed positional tax per loop, 30–45× the marginal gain after loop 2; sequential looping has no such term. It is the **price of parallelization**, not a property of looping.

Insight 3: Loop granularity is getting finer and converging on the middle

From UT's whole-model loop, to Huginn's three-stage design (ends unlooped), to Loopie/Training-Free/SMELT's per-layer or mid-stack loops, to MixerLoop looping only the sequence mixer while the FFN runs once (retaining 41.5% of the gain at 45.9% fewer looped FLOPs), granularity keeps shrinking. Three independent lines point to the same placement rule: Training-Free's depth-fraction rule (optimal window centered at fractional depth 0.43–0.71, mode 0.5), SMELT's "middle half", and Huginn's design rationale that early and late layers differ functionally from the homogeneous middle. Ends embed and read out and are specialized; the middle is homogeneous and reusable.

For MoE there is a hard reason for layer-loop: block-loop routes to different experts on each iteration (routing thrash). Training-Free (no training) and Loopie (from scratch) agree independently.

Insight 4: ODE flow or fixed-point iteration? Two philosophies are diverging

Training-Free's finding is that the mid-stack of a pretrained transformer "is not contractive in any useful sense": Anderson, heavy-ball, and Aitken accelerators all underperform simple RK sub-stepping. For a **trained** model the right loop refines the integral over [0,1] (damped Euler $x \leftarrow (1-1/K)x+(1/K)g(x)$) rather than advancing to t=K.

The other branch **trains** the loop to be a fixed-point iteration: STARS via Jacobian spectral-radius regularization, Fixed-

Point Reasoners via convergence-as-halting, Attractor Models via implicit differentiation. The motivation is stability: Viakhirev et al. classify operators as settling / marginal / drifting; settling operators never degrade with depth, and a single terminal fixed-point objective turns drifting into settling. Huginn is non-settling.

The two philosophies fit two situations: follow the ODE flow when retrofitting an existing model; design stability into the objective when training from scratch. LoopCoder-v2's oscillating update directions ($\cos\theta<0$) are exactly non-settling behavior.

Insight 5: Training stability went from folklore to theorems

Huginn's two failed runs were lessons in 2025; in 2026 they are theorems: residual scaling needs $1/N$ not $1/\sqrt{L}$ (Wang et al.), the DeepNorm exponent must rise from 1/4 to 1/2 (DeepLoop), and the optimal LR depends only on the number of unique layers and transfers across loop counts. In practice, Fully Looped Transformer trains stably to 12 loops and LoopMTP to 15. With SmallInit and sandwich normalization, the "looped models are hard to train" obstacle standing since 2018 is essentially dismantled.

Insight 6: Inference cost is the last big bottleneck, now decomposed into three sub-problems

Looped inference costs three things: latency (serial loops), KV memory (one copy per loop), and quantization fragility. Each has 2025–2026 solutions: latency via PLT's parallelization or Geiping's diffusion-style parallel sampler (5×); KV via sharing (Huginn, MoR, MELT) or compression (Looped Latent Attention finds cross-loop KV low-rank, 32× near-lossless); quantization via LoopQ. These are independent and stackable.

One accounting caveat: Loopie's "compute-matched" is matched training step time; at inference it executes 54 block passes per token versus the reference's 48 with a larger KV cache, and the paper reports no inference latency. **A win at matched training compute is not automatically a win at matched inference cost.**

Insight 7: Latent looping and explicit CoT are complements, not substitutes

Three pieces of evidence: LoopCoder-v2's Thinking mode on top of R=2 gives LiveCodeBench +26.9, beyond the sum of the parts; LOTUS supervises latent positions with gold CoT tokens and is the first to match explicit CoT at 3B while cutting thought latency 2.5–6.9×; Xu & Sato's formal separation shows each excels on a different problem class.

The companion finding is that **standard RL is mismatched to looped models**: GRPO credits only the final latent state (RLTT identifies this as why RL on looped models kept failing), and distributing reward along the latent trajectory gives Ouro-2.6B +10.9% on math; LoopRPT assigns RL signal to latent steps already in pretraining. Post-training a looped model needs redesigned objectives; recycling CoT-era RL recipes wastes the loop.

Insight 8: The capability profile is now clear — loops manipulate knowledge, they do not store it

Ouro's controlled experiments attribute its advantage to knowledge manipulation, not capacity; Huginn is weak closed-book and strong open-book (OBQA +11.0); Frey et al. find looping helps math while memory banks help commonsense, and only the combination beats an iso-FLOP baseline; Dual-Path runs a "deep" looped sublayer and a "wide" FFN in parallel and learns gates that send punctuation, symbols, and arithmetic tokens deep while function words and lexical content go wide.

This defines the loop's best partners: **retrieval, long context, MoE expert banks** — anything that externalizes storage. Loopie and SMELT choosing MoE backbones, and Adaptive Loops and Memory adding gated memory banks, follow this logic.

Insight 9: Applications are expanding from math puzzles to agents and embodiment

Evidence in 2025 concentrated on Sudoku, mazes, ARC, GSM8K; 2026 adds three arenas: (1) **software-engineering agents** — LoopCoder-v2's SWE-bench Verified +21.4 and large Terminal-Bench gains; (2) **tool calling** — Popescu et al. find multi-step, dependency-aware tool use rises with recurrent depth, and Nanbeige4.2-3B beats Qwen3.5-9B on agentic benchmarks at 3B; (3) **robotics** — RD-VLA's looped action head takes some tasks from 0% to >90% and runs 80× faster than token-reasoning VLAs. The common thread: these tasks require maintaining intermediate state and dependencies within one forward pass, which is what loops are for.

Insight 10: Productization has started; the toolchain has not caught up

Five open looped-model families exist — Huginn, Ouro, LoopCoder-v2, Loopie, Nanbeige4.2-3B — and Nanbeige4.2-3B was trained on 28T tokens and shipped as a product. Yet Halloran's replication report finds five out-of-the-box bugs running it in Hugging Face transformers (including a silently zeroed RoPE buffer and calls to removed cache APIs), and even after fixes, layer reuse doubles attention memory. Inference frameworks, quantization tools, and KV management have no standard interface for looped models. Loopie shipping megatron-loopie and vllm-loopie branches alongside the paper is a step in that direction.

7. Inferences about GPT-6 Astra (explicitly labeled as inference)

The following rest on the public literature surveyed here and on **no** first-hand information about Astra.

1. If Astra uses recurrent depth, the recipe in the public literature closest to "frontier scale + compute-matched" is the Loopie/SMELT line: MoE backbone, mid-stack or per-layer looping, small loop count. The Huginn line of looping a whole model 32 times has no compute-matched evidence at frontier pretraining budgets.
2. The common speculation that "OpenAI found a way around two-loop saturation" starts from a false premise — two-loop saturation is PLT-specific. Sequential looped models (Huginn) never had that bottleneck; the real one is inference cost, which public methods such as 32× KV compression and 5× diffusion-style parallel sampling have already cut by an order of magnitude.
3. The "security concerns" mentioned in The Information's report match a real issue in the literature: latent reasoning produces no readable intermediate tokens, and interpretability work (Lu 2025, Jacobian Lens 2026) shows that probes of looped models' internal states depend on layer and decoding method — supervising the reasoning process is harder than supervising CoT.
4. On timing, Loopie (July), Looped Latent Attention (July), SMELT (1 September), and Astra (early September) cluster tightly; the more reasonable reading is that several groups pushed this direction to usability in the first half of 2026 simultaneously, not that one paper caused one product.

8. Open problems

1. **Compute-matched multi-loop.** Every compute-matched win is at R=2. Whether R≥4 can win under matched pretraining compute has no public evidence.
2. **The full inference bill.** Training step time, inference latency, and peak KV memory need to be reported together; no paper yet does all three.
3. **Scaling laws for looped models.** SMELT fits to 54B non-embedding parameters, Schwethelm to ~1B. Whether the functional form (Chinchilla plus an r^{φ} term) holds at 100B+ is unknown.
4. **Adaptive halting at pretraining scale.** MoR, TaH, and RecurTrace results are all ≤8B; Ouro's entropy-regularized depth allocation is the only one validated at 7.7T tokens.
5. **Supervisability of latent reasoning.** LOTUS shows gold-CoT supervision of latent positions works, but requires CoT data; making latent trajectories readable and auditable without it is an open safety question.
6. **The mechanism of layer-loop.** Of Loopie's three reasons (scaling, infrastructure, local sharing) only scaling has experiments; SMELT's "the second visit reduces the attention sink" is the only mechanistic account so far.
7. **Looping versus sequence-length scaling.** Hidden Decoding (WeLM 617B) adds compute along the sequence dimension and claims better pipeline-parallel compatibility; Loopie's layer-loop was designed precisely for pipeline compatibility. No head-to-head at 100B+ exists.

9. Conclusion

In eight years recurrent depth went from an encoder–decoder trick for bAbI overfitting to a candidate component of frontier architectures. Three turns in the first half of 2026 — training-free looping giving existing models 2 free points, PLT keeping latency flat as loops grow, and Loopie/SMELT winning at matched compute for the first time — pushed it past the engineering-feasibility threshold. What remains is a full accounting of inference cost, compute-matched evidence for more than two loops, and supervisability of latent reasoning. For researchers the most valuable gaps are compute-matched scaling laws at R≥4 and loop-specific RL objectives; for engineering teams the practical conclusion is: loop the middle, twice, per layer, on an MoE backbone — and measure latency and KV before shipping.

Appendix A: Circulation errors corrected in this report

Common claim	What the paper says
Huginn crushes its baseline “at the same 0.8T tokens”	The non-recurrent baseline trained for 0.18T; at equal 0.18T the looped model gets ARC-E 53.62 vs 46.42
“Two loops suffice, three regress” is a law of looped transformers	It is specific to PLT (cross-loop position offset); Huginn improves monotonically to r=32
Loopie and Qwen3-30B-A3B use “the same compute”	The same measured optimizer-step time; Loopie’s nominal FLOPs are 42% higher and inference cost is unreported
Training-Free makes open models “stronger”	Gains of about 1–2.6 points, concentrated on knowledge MC; small distilled models can lose points
Huginn “performs like a 50B model at 3.5B”	FLOP-equivalent, not benchmark-equivalent; on benchmarks it is near OLMo-7B and behind OLMo-2

Appendix B: Method and sources

- Five core papers: arXiv PDFs (`papers/original/`) and arXiv HTML full texts (`papers/src/`), read section by section.
- Chinese translations: SuperTranslate layout-preserving translation with object-level QA (`papers/zh/`; QA record in `QA_SUMMARY.md`).
- Related literature: arXiv API searches on 2026-09-06 combining looped/recurrent-depth/depth-recurrent transformer, latent reasoning □ (loop|recurrence|recursive), and named models; 60 deduplicated abstracts kept in `data/related_abstracts.json`.
- Writing rules: lead with the claim, state necessary limits once, make every number checkable; cite no primary source we could not access.

Appendix C: Glossary

Term	One-line meaning
Recurrent depth / looped transformer	The same parameters applied repeatedly along depth; effective depth = stored layers × loop count
model-loop / block-loop / layer-loop	Loop the whole stack / only a middle segment / each layer several times before the next (Figure 1)
Input injection	Re-feed the input embedding at every loop; otherwise the iteration is unstable
Truncated backprop	Gradients through only the last k loops so memory is independent of loop count
Adaptive halting (ACT)	Each token decides how many loops it needs
Path independence	Different random initial states converge to the same behavior
KV cache	Cached attention keys/values; one extra copy per loop, the main memory bottleneck
MoE	Mixture of experts; “20B-A2B” = 20B total, 2B active parameters
FLOP-matched vs measured-compute-matched	Aligning theoretical compute vs measured training step time; looping’s activation-memory savings only appear in the latter
Latent reasoning vs explicit CoT	Extra computation in hidden space vs intermediate steps written as tokens
Fixed point	A state another loop leaves unchanged; one line of work trains loops to converge to fixed points
PLT / CLP	Parallel Loop Transformer / cross-loop position offset
φ in the iso-depth scaling law	How many unique parameters one recurrence is worth: 1 = full equivalence, 0 = no capacity gain; measured 0.46

Report 01 · Universal Transformers: Where Recurrent Depth Began

Item	Detail
Paper	Universal Transformers
arXiv	1807.03819 (Jul 2018), ICLR 2019
Authors	Mostafa Dehghani*, Stephan Gouws*, Oriol Vinyals, Jakob Uszkoreit, Łukasz Kaiser (U. Amsterdam / DeepMind / Google Brain)
Code	tensor2tensor
In this repo	Original `papers/original/1807.03819.pdf` · Chinese `papers/zh/1807.03819.zh.pdf`

At a glance

Problem · Transformers have fixed depth and lack the RNN bias toward re-applying one transformation; they overfit on multi-hop reasoning (bAbI) and fail on algorithmic tasks.
Method · Replace N distinct layers with one layer applied T times, add position+timestep coordinate embeddings; ACT lets each position decide when to stop.
Result · bAbI joint-training error 22.1 → 0.29, LAMBADA perplexity 7321 → 142, WMT14 +0.9 BLEU.
Limits · Poor parameter-to-compute ratio, unstable training, inference cost undiscussed — the research agenda of the next eight years.
Plain words · Run the same circuit several times: hard tokens get more rounds, easy ones stop early.



Figure (labels in Chinese): one shared block repeated over time steps; positions halt at different steps.

One sentence

Replace the Transformer's N distinct layers with one set of parameters applied T times, add a per-position halting mechanism (ACT) that decides how long each symbol "thinks", and you get a parallel-in-time, recurrent-in-depth, Turing-complete sequence model that beats a same-size Transformer on bAbI, LAMBADA, algorithmic tasks, and WMT14 translation.

1. The problem

The 2018 Transformer had two weaknesses that were already visible at the time. First, fixed depth: every token goes through the same 6 layers regardless of difficulty. Second, it lacks the RNN's inductive bias toward applying the same transformation repeatedly, so it does badly on algorithmic tasks (copy, reverse, addition) and simply overfits on multi-hop reasoning benchmarks like bAbI (the authors note they tried many hyperparameters and sizes; it always overfit).

Neural GPUs and Neural Turing Machines have recurrence and universal computation, but cannot compete with the Transformer on real language tasks such as translation. UT's goal is to put both sets of strengths into one model.

2. Method

2.1 Recurrence over depth

Starting from input embeddings $H^0 \in \mathbb{R}^{m \times d}$, each time step t does two things for all m positions in parallel: multi-head self-attention exchanges information across positions, then a position-wise transition function transforms each position:

$$A^t = \mathrm{LN}((H^{t-1} + P^t) + \mathrm{MHSA}(H^{t-1} + P^t)), \qquad H^t = \mathrm{LN}(A^t + \mathrm{Transition}(A^t))$$

After T steps the encoder outputs H^T. All time steps share the same attention and transition parameters, so "number of layers" no longer equals "number of parameters". The transition function is either a two-layer position-wise feed-forward network or a depthwise-separable convolution (the latter works better on algorithmic tasks).

2.2 Coordinate embeddings: position + time step

Because parameters are shared, the model must know which step it is on. UT generalizes sinusoidal position encodings into two-dimensional coordinate embeddings P^t: sinusoidal encodings of position i and time step t are computed separately and summed (Eq. 6–7), then added back to the state at every step. This is the ancestor of every later "timestep encoding" in looped models.

2.3 Dynamic halting (ACT)

Graves' (2016) Adaptive Computation Time is attached to each position: at every step the model predicts a halting probability per position; once the cumulative probability crosses a threshold, that position stops updating and its state is copied forward until all positions halt or the step limit is hit. Different tokens therefore receive different "ponder time".

2.4 Computational power

With fixed T, UT is a Transformer with weights tied across layers. Letting T scale with the input length, UT reduces exactly to a Neural GPU (set self-attention to identity, the transition to a convolution, T = input length), and is therefore Turing-complete given enough memory. The authors stress the key difference from RNNs: an RNN's recurrent step can only access a fixed-size state vector, whereas each UT step attends to every position's representation from the previous step.

3. Key experiments and numbers

Task	Transformer	UT	UT + dynamic halting
bAbI 10K, 20 tasks joint (avg error / failed tasks)	22.1 (12/20)	0.47 (0/20)	**0.29 (0/20)**
bAbI 1K, joint	26.8 (14/20)	8.50 (8/20)	**7.78 (5/20)**
Subject–verb agreement, 5 attractors	0.883	0.892	**0.907**
LAMBADA LM test perplexity	7321	319 (fixed 6 steps)	**142**
WMT14 En–De BLEU (base)	28.0	**28.9**	–
Copy / Reverse / Addition (train len 40, test len 400), char acc.	0.53 / 0.13 / 0.07	**0.91 / 0.96 / 0.34**	–
Learning-to-Execute memorization (Copy/Double/Reverse), seq acc.	0.63 / 0.55 / 0.26	**1.0 / 1.0 / 1.0**	–

Three observations worth isolating:

1. **Ponder time tracks difficulty.** On bAbI questions needing 1, 2, and 3 supporting facts, mean ponder time is 2.3±0.8, 3.1±1.1, and 3.8±2.2 steps. The model allocates compute on demand.
2. **Dynamic halting acts as a regularizer.** On LAMBADA the halting model averages 8.2 steps; fixed 8- and 9-step UTs both beat the 6-step model but neither beats the dynamic one. Taking more steps is not the whole story.
3. **More distractors, bigger advantage.** In subject–verb agreement, UT+ACT's margin over the prior best grows monotonically with attractor count (from 0 to +0.027). The gains from recurrence concentrate on hard instances — a pattern that reappears almost verbatim in the Huginn paper seven years later.

4. Why it matters

UT fixed the three ingredients every later work on this line uses: **parameter-shared depth recurrence**, **a conditioning signal re-injected at every step** (coordinate embeddings here, input injection later), and **adaptive halting**. The "model-loop" of today's recurrent-depth literature is UT's original form.

It also left three problems open that define the following seven years of research:

- **Parameter-to-compute ratio.** Sharing parameters collapses parameter count at a given hidden size, which hurts on parameter-dominated tasks like language modeling. UT base beats Transformer base by only 0.9 BLEU on translation — in parameter-rich regimes the marginal value of recurrence is small. This was not properly addressed until MoEUT (2024) and Loopie (2026) combined looping with MoE.
- **Training instability.** ACT's halting probabilities need extra regularization and are hyperparameter-sensitive; deep recurrence is hard to optimize. Huginn later used truncated backpropagation, Loopie used layer-loop, and several 2026 papers derive residual-scaling theory to fix this.
- **Compute cost.** Every extra loop is another full forward pass. UT does not discuss inference latency or the KV cache; those become the subject of PLT, MELT, and Looped Latent Attention in 2025–2026.

5. Limitations

- Small scale (WMT base at most); nothing above ~100M parameters.
- ACT gains are inconsistent: the translation experiments use fixed steps, not ACT.
- Encoder–decoder; the decoder-only era needed a redesign (Huginn's prelude–core–coda is the direct descendant).

6. Relation to other work

- **Predecessors:** Neural GPU (Kaiser & Sutskever 2016), Neural Turing Machine (Graves 2014), ACT (Graves 2016).
- **Contemporaries:** ALBERT (Lan et al. 2019) uses cross-layer sharing on BERT for compression — same idea, different goal.
- **Direct descendants:** Sparse Universal Transformer (Tan et al. 2023) and MoEUT (Csordás et al. 2024) address the parameter–compute ratio; Giannou et al. (2023) re-prove universality from a "programmable computer" angle; Geiping et al. (2025) scale it to 3.5B.

7. Practical takeaways

- Recurrence pays off on tasks that need iteration (multi-hop, algorithmic, long dependencies) and barely at all on pure recall. Split benchmarks along this line when evaluating looped models.
- Adaptive halting is a regularizer, not only a compute saver. RecurTrace and Think-at-Hard (2026) re-confirm this.
- Timestep conditioning (telling the model which step it is on) is in tension with path independence: the Huginn paper reports that feeding the step index breaks extrapolation. UT's choice is not right in every setting.

Appendix: quick numbers

- Encoder–decoder; parameters shared across time steps; transition = FFN or separable conv.
- bAbI 10K joint error: 22.1% → 0.29%.
- LAMBADA test perplexity: 7321 → 142.
- WMT14 En–De: 28.0 → 28.9 BLEU.
- ACT mean steps vs. required facts: 2.3 / 3.1 / 3.8 (1 / 2 / 3 facts).

Glossary

Term	One-line meaning
Recurrent depth / looped transformer	The same parameters are applied repeatedly along depth; effective depth = stored layers × loop count, parameters unchanged
model-loop / block-loop / layer-loop	Loop the whole stack / only a middle segment / each layer several times before the next; see `assets/figures/diagrams/loop_granularity.png`
Input injection	Re-feed the input embedding at every loop; without it the iteration is unstable
Truncated backprop	Compute gradients through only the last k loops so memory is independent of the loop count
Adaptive halting (ACT)	Let each token decide how many loops it needs

Term	One-line meaning
Path independence	Different random initial states converge to the same behavior
KV cache	Cached attention keys/values; looped models add one copy per loop, the main memory bottleneck
MoE	Mixture of experts: each token activates a small subset of parameters; “20B-A2B” = 20B total, 2B active
FLOP-matched vs measured-compute-matched	Aligning theoretical compute vs aligning measured training step time — the activation memory looping saves only shows up in the latter
Latent reasoning vs explicit CoT	Extra computation in hidden space vs intermediate steps written as text tokens
Fixed point	A state that another loop leaves unchanged; one line of work trains loops to converge to fixed points for stability
PLT / CLP	Parallel Loop Transformer / cross-loop position offset — shift the previous round’s states by one token so rounds run in parallel

Report 02 · Scaling up Test-Time Compute with Latent Reasoning: Recurrent Depth at 3.5B

Item	Detail
Paper	Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach
arXiv	2502.05171 (Feb 2025)
Authors	Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Tom Goldstein (ELLIS Tübingen / MPI-IS / U. Maryland / LLNL)
Model / code	huginn-0125 · recurrent-pretraining
In this repo	Original `papers/original/2502.05171.pdf` · Chinese `papers/zh/2502.05171.zh.pdf`

At a glance

Problem · Can the 2018 recurrent-depth idea be trained at modern LLM scale, and does it really improve with more test-time loops?
Method · 2-layer prelude + 4-layer looped core + 2-layer coda; loop count sampled per sequence at training (mean 32), backprop through the last 8 steps only; 800B tokens on 4096 GPUs.
Result · Loops 4 → 32: ARC-E 49.1 → 69.9, GSM8K CoT ≈0 → 42.1; at equal 0.18T tokens the fair baseline comparison gives ARC-E +7.2 and ~5× on GSM8K.
Limits · Every extra loop is another core forward; weak factual memory (MMLU 31); a single large run whose LR was never cooled.
Plain words · Few parameters, but the model can think several rounds before speaking; the harder the question, the more the rounds help.

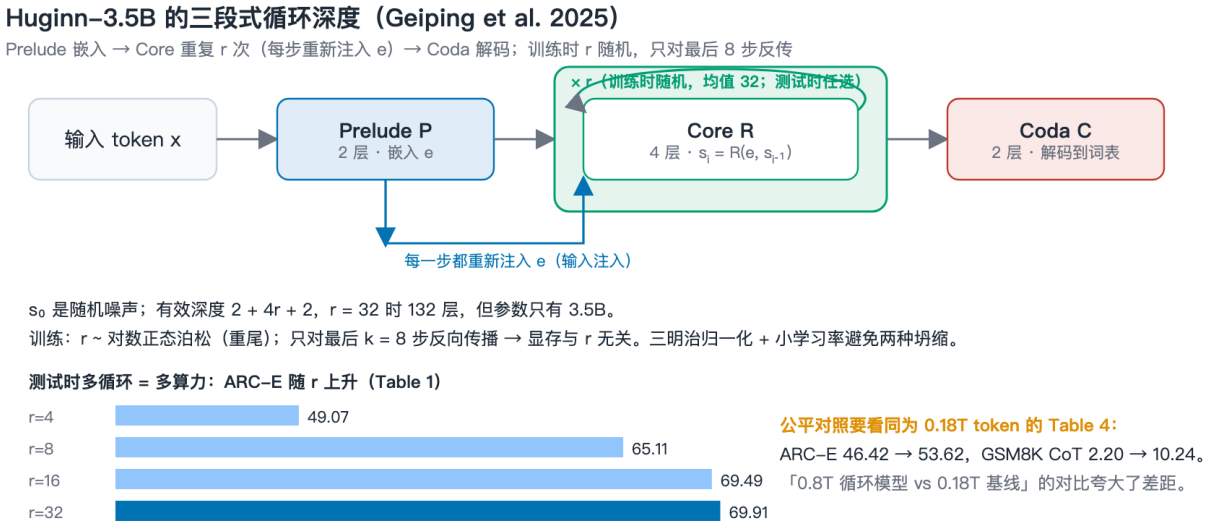


Figure (labels in Chinese): the three-stage architecture and training recipe; bottom-right, ARC-E rising with loop count and the correct reading of the fair comparison.

One sentence

Pretrain a 3.5B-parameter decoder-only model with a 2-layer prelude, a 4-layer recurrent core, and a 2-layer coda (Huginn-0125) on 800B tokens across 4096 AMD MI250X GPUs, sampling the number of loops randomly during training and backpropagating through only the last 8; at inference, raising the loop count from 4 to 32 lifts ARC-E from 49.1 to 69.9 and GSM8K-CoT from near zero to 42.1, with no chain-of-thought data at all. This paper coined "recurrent depth" and is the first to make it train at modern language-model scale.

1. The problem

Mainstream test-time scaling (o1, R1) generates longer chains of thought. The costs: purpose-built long-CoT training data, large context windows, and every intermediate thought squeezed into discrete tokens. The authors test the alternative — let

the model iterate in continuous latent space by re-applying one block, unrolling the network to arbitrary depth.

UT proposed this in 2018 but nobody had trained it at modern LLM scale. The question here is engineering: which architecture, initialization, and objective let depth recurrence converge stably at billions of parameters and nearly a trillion tokens, and actually improve with more loops.

2. Method

2.1 Three-stage architecture: Prelude → Core (looped) → Coda

With r recurrences and input tokens $\mathbf{x}$:

$$\mathbf{e} = P(\mathbf{x}), \qquad \mathbf{s}_0 \sim \mathcal{N}(0, \sigma^2 I), \qquad \mathbf{s}_i = R(\mathbf{e}, \mathbf{s}_{i-1}) \; (i = 1..r), \qquad \mathbf{p} = C(\mathbf{s}_r)$$

- **Prelude** P (2 layers) embeds tokens into latent space.
- **Core** R (4 layers) takes the concatenation of the previous state $\mathbf{s}_{i-1}$ and the embedding $\mathbf{e}$, maps it back to width h through an adapter $A : \mathbb{R}^{2h} \to \mathbb{R}^h$, and runs 4 transformer layers. **e is re-injected at every step.** The authors' analogy is gradient descent on $E(\mathbf{x}, \mathbf{y})$: every step needs the data $\mathbf{y}$; an iteration that sees the data only once cannot be stable. The initial state $\mathbf{s}_0$ is random noise; together with input injection this drives convergence to a steady state independent of initialization (path independence).
- **Coda** C (2 layers) decodes to the vocabulary.

Main model shape $(l_P, l_R, l_C) = (2, 4, 2)$, width $h = 5280$, 55 heads × 96, MLP inner 17920. Parameters: 1.5B non-recurrent, 1.5B recurrent core, 0.5B tied embeddings — 3.5B total. At $r = 32$ the effective depth is $2 + 4 \times 32 + 2 = 132$ layers.

2.2 Micro design: sandwich normalization

Each layer uses four RMSNorms bracketing attention and MLP (n_1–n_4). At small scale pre-norm and post-norm both train; at large scale only the sandwich format produces a model that actually uses its recurrence (see the two failed runs in §4). Everything else is standard: RoPE (base 50000), gated SiLU MLP, QK biases.

2.3 Objective: random unrolling + truncated backprop

For each training sequence sample $r \sim \Lambda$ and minimize the expected loss

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}} \, \mathbb{E}_{r \sim \Lambda} \, L(m_\theta(\mathbf{x}, r), \mathbf{x}')$$

Λ is a log-normal Poisson: $\tau \sim \mathcal{N}(\log \bar{r} - \frac{1}{2}\sigma^2, \sigma)$, $r \sim \text{Poisson}(e^\tau) + 1$, with mean $\bar{r} = 32$ and $\sigma = 1/2$. It mostly samples below 32 but has a heavy tail.

Backpropagate through the last $k = 8$ iterations only. Activation memory and backward compute become independent of r, which is what makes the heavy-tailed distribution affordable. The prelude still receives gradients at every step because its output $\mathbf{e}$ is injected at every step. This is truncated BPTT, applied along depth instead of time.

2.4 Large-scale training details

- Data: 800B tokens (795B realized), all public, skewed toward code and math; instruction data mixed directly into pretraining. Vocabulary 65536, sequence length 4096.
- Hardware: ORNL Frontier, 4096 MI250X, 21 segments of up to 12 hours; 108.75 TFLOP/s per node (87% achievable utilization), 52–64 TFLOP/s per GPU at 4096 GPUs. Because of weight sharing, even the largest model trains with data parallelism plus optimizer sharding only — no tensor parallelism.
- Optimization: Adam (0.9, 0.95), constant LR 4×10^{-5} after a 4096-step warm-up, no decay; one r per micro-batch synchronized across workers (locked-step sampling) so nobody idles.
- Initialization: Takase et al. (2024), variance $2/(5h)$; output projections $1/(5hl)$ with $l = 132$ effective layers.

3. Key experiments and numbers

3.1 Performance rises monotonically with loop count (Table 1)

Loops r	ARC-E	ARC-C	HellaSwag	MMLU	OBQA	PiQA	SciQ	WinoGrande
4	49.07	27.99	43.46	23.39	28.20	64.96	80.00	55.24
8	65.11	35.15	58.54	25.29	35.40	73.45	92.10	55.64
16	69.49	37.71	64.67	31.25	37.60	75.79	93.90	57.77
32	**69.91**	**38.23**	**65.21**	**31.38**	**38.80**	**76.22**	93.50	**59.43**

Against other open models it lands near OLMo-7B (2.5T tokens) and behind OLMo-2 (4T tokens, curated data). The authors' own framing: a 3.5B model that spends FLOPs like a 32B one — per-token pretraining FLOPs are close to a 32B dense transformer, and test-time scaling continues up to the FLOP budget of a ~50B model.

3.2 Math and code (Tables 2–3)

GSM8K CoT 34.80/42.08 (with system prompt, flexible/strict), Minerva MATH 12.58, MBPP 24.80, HumanEval 23.17. It beats every comparison model except OLMo-2 on math and every general-purpose open model on code. The constant LR allows weight averaging to simulate a cooldown: the EMA model reaches 47.23% GSM8K flexible at $r = 64$.

3.3 The fair comparison: same data, same setup, non-recurrent baseline (Table 4)

This is the most important table in the paper and the one most often misquoted downstream. The baseline is **the same architecture run once through the core, trained on 180B tokens**; the recurrent model also has a 180B early checkpoint.

Model	Tokens	ARC-E	HellaSwag	GSM8K CoT
Fixed-depth baseline	0.18T	46.42	37.34	1.82/2.20
Recurrent, early ckpt, $r = 32$	0.18T	53.62	48.80	9.02/10.24
Recurrent, early ckpt, $r = 1$	0.18T	34.01	29.19	0.00/0.15
Recurrent, final, $r = 32$	0.8T	69.91	65.21	34.80/42.08
Recurrent, final, $r = 1$	0.8T	34.89	29.34	0.00/0.00

Correct reading: **at equal 180B tokens**, the recurrent model beats the baseline by 7.2 points on ARC-E, 11.5 on HellaSwag, and roughly 5× on GSM8K. Some popular write-ups place the 0.8T recurrent model next to the 0.18T baseline as if both were "0.8T", which inflates the gap; the paper has no 0.8T non-recurrent baseline.

The $r = 1$ rows carry a second message: from 180B to 800B tokens, $r = 1$ performance barely moves (34.01 → 34.89). Everything learned in the last 620B tokens lives **entirely in the iterations of the recurrent block**; the prelude and coda did not get stronger.

3.4 Where recurrence helps most

- Hard tasks benefit most (ARC-C); pure fact recall benefits least (SciQ).
- **More context, more loops needed.** ARC-C saturates at 8–12 iterations zero-shot, ~20 with 1 example, 32 with 25–50 examples (Figure 9). The model spends more recurrence to extract more from more context.
- **Open-book vs. closed-book.** Given the relevant fact, OBQA rises from 38.2 to 49.2 (+11.0), nearly closing the gap to OLMo-2. Recurrent models memorize fewer facts but reason more over what is in context.

4. Two failed runs: why scaling is hard (§4.3)

- **Failure 1:** parameter-free RMSNorm, no embedding scale, additive adapter $A(\mathbf{s}, \mathbf{e}) = \mathbf{s} + \mathbf{e}$, peak LR 4×10^{-4}. **Hidden-state collapse**: the correlation of hidden states across tokens shoots to 1.0 — the model predicts the same state for every position. Each recurrence mixes the sequence a little more until it collapses.
- **Failure 2:** add embedding scaling, revert to conventional pre-norm, learned adapter. Correlation spikes to 1.0 in the first 150 steps then recovers, but **recurrence collapse** follows: validation perplexity is identical at $r = 1$ and $r = 32$. The model learned early to ignore the incoming state **s** and sits in a no-recurrence local minimum.
- **Success:** back to sandwich norms, LR down 10× to 4×10^{-5}. The next 750B tokens train smoothly without loss spikes.

These lessons were later formalized by Loopie (SmallInit), DeepLoop, and "On the Residual Scaling of Looped Transformers" ($\varepsilon = 1/N$ scaling).

5. Capabilities that come for free (§6)

Each of these needs dedicated training on a standard transformer; Huginn gets them zero-shot:

1. **Per-token adaptive compute.** Stop when the KL between two successive steps' output distributions drops below 5×10^{-4}. Steps needed differ by MMLU category: fewest for high-school math, 3.5 more on average for moral scenarios. MT-Bench 5.63 → 5.56.
2. **KV-cache sharing.** Keep only k recurrence slots per token; step i reads/writes slot i mod k. With budget 4, MT-Bench is 5.86 — no loss. All recurrent KV entries come from the same projection matrices, so they "match".
3. **Continuous chain of thought.** Warm-start $\mathbf{s}_0$ from the previous token's final state $\mathbf{s}_r$ instead of fresh noise; converges 1–2 steps sooner on average. Related to Coconut (Hao et al. 2024), except Huginn pretrains recurrence from scratch and needs no CoT data.
4. **Self-speculative decoding.** Draft with few loops, verify with many; the drafting states are reused during verification, so no separate draft model is needed.

6. What happens in latent space (§7)

PCA of trajectories $\{\mathbf{s}_i\}$ (Figures 11–12):

- Most tokens converge monotonically to a fixed point.
- Tokens tied to numerical reasoning (the "3" in a GSM8K problem) fall into **multi-dimensional orbits**, periodic in all three pairs of leading principal directions; structural tokens ("makes", "thinks") also orbit.
- Some key tokens ("wrong") behave as **sliders**: steady drift along one direction, plausibly implementing a step counter.
- Convergence speed is context-dependent: three identical "." tokens in one sentence behave differently; the crux of a question and the start of the answer are "deliberated" longest.
- Path independence holds: different $\mathbf{s}_0$ give the same orbits, fixed points, and drifts.

None of this geometry is in the training objective; it emerges from scale. The authors read it as the model organizing computation spatially in a high-dimensional space rather than linearly as in verbal CoT.

7. Limitations

- One large run: 47000 steps, LR never cooled, data mixture never ablated. The authors call it a proof of concept.
- Each extra loop is another core forward; $r = 32$ costs 16×+ the inference of an 8-layer dense model, and the KV cache grows linearly in r (§5's sharing mitigates, not solves).
- Weak factual memory (MMLU 31.4, low closed-book OBQA), consistent with the few-parameters-many-FLOPs design.
- Lu et al. (2025) probe Huginn with logit lens and find limited evidence of interpretable "latent CoT", with diminishing returns from extra loops. Viakhirev et al. (2026) classify Huginn as "non-settling" — more loops do not guarantee the decoded answer stays fixed.

8. Relation to other work

- Roots: UT (2018); deep-thinking line (Schwarzschild 2021, Bansal 2022: random unrolling + input injection); path-independent equilibrium models (Anil 2022).
- Versus DEQ: DEQ solves for the fixed point directly; Huginn trains by truncated unrolling, which the authors argue scales better.
- Direct successors: Ouro (2.6B, 7.7T tokens, 4 loops), Loopie (MoE, 2-step layer-loop), LoopCoder-v2 (PLT, 2 loops); Geiping's group later connects recurrent-depth models to diffusion LMs with a parallel sampler giving up to 5× speedup (2510.14961); McLeish et al. (2025) study converting pretrained non-recurrent models into recurrent ones.
- §9's prediction has come true: recurrence (good at reasoning) and MoE (good at storage) are complementary and "a future architecture would contain both" — exactly Loopie and SMELT.

9. Practical takeaways

- Normalization and initialization look irrelevant at small scale and decide success at large scale. Any small-scale ablation of a looped architecture must check whether the loop is actually used (compare perplexity at $r = 1$ and $r = \max$).
- Input injection + random initial state + random loop count + truncated backprop is the current standard recipe for extrapolating recurrent models.
- Recurrent models trade parameters for compute: strong at reasoning over context, weak at storing facts. They gain more than dense models from retrieval or long context at deployment.
- "3.5B behaves like 50B" means FLOP-equivalent, not benchmark-equivalent. The gap to OLMo-2-7B is real.

Appendix: quick numbers

- Shape (2,4,2), $h = 5280$, 3.5B params, 132 effective layers at $r = 32$.
- 800B tokens, 4096 MI250X, constant LR 4×10^{-5}, $\bar{r} = 32$, truncated backprop $k = 8$.
- r: 4→32 gives ARC-E 49.07→69.91, HellaSwag 43.46→65.21, GSM8K CoT (strict) →42.08.
- Fair 180B comparison: ARC-E 46.42 vs 53.62; GSM8K CoT 2.20 vs 10.24.
- Adaptive exit threshold KL $< 5 \times 10^{-4}$; KV budget 4 keeps MT-Bench intact.

Glossary

Term	One-line meaning
Recurrent depth / looped transformer	The same parameters are applied repeatedly along depth; effective depth = stored layers × loop count, parameters unchanged
model-loop / block-loop / layer-loop	Loop the whole stack / only a middle segment / each layer several times before the next; see `assets/figures/diagrams/loop_granularity.png`
Input injection	Re-feed the input embedding at every loop; without it the iteration is unstable
Truncated backprop	Compute gradients through only the last k loops so memory is independent of the loop count
Adaptive halting (ACT)	Let each token decide how many loops it needs
Path independence	Different random initial states converge to the same behavior
KV cache	Cached attention keys/values; looped models add one copy per loop, the main memory bottleneck
MoE	Mixture of experts: each token activates a small subset of parameters; "20B-A2B" = 20B total, 2B active
FLOP-matched vs measured-compute-matched	Aligning theoretical compute vs aligning measured training step time — the activation memory looping saves only shows up in the latter
Latent reasoning vs explicit CoT	Extra computation in hidden space vs intermediate steps written as text tokens
Fixed point	A state that another loop leaves unchanged; one line of work trains loops to converge to fixed points for stability
PLT / CLP	Parallel Loop Transformer / cross-loop position offset — shift the previous round's states by one token so rounds run in parallel

Report 03 · Training-Free Looped Transformers: A Loop Wrapper for Frozen Checkpoints

Item	Detail
Paper	Training-Free Looped Transformers
arXiv	2605.23872 (May 2026)
Authors	Lizhang Chen*, Jonathan Li*, Chen Liang, Ni Lao, Qiang Liu (UT Austin)
In this repo	Original `papers/original/2605.23872.pdf` · Chinese `papers/zh/2605.23872.zh.pdf`

At a glance

Problem · Can a released, frozen checkpoint get stronger by looping at inference alone — no weight change, no training?
Method · Treat the middle 4 layers as one ODE segment: do not run it K times, split it into K damped sub-steps (Runge–Kutta style with an anchor β); use layer-mode on MoE to pin routing.
Result · 7 model families, 45 (model, benchmark) cells, 87% non-negative; Qwen3-4B MMLU-Pro +2.64, GPQA +2.01, Qwen1.5-MoE ARC-C +2.30.
Limits · Gains cap at ~2–3 points and concentrate on knowledge MC; sub-3B distilled models can lose; 15–30% extra latency.
Plain words · Walk the same stretch in smaller steps and you land more precisely — but do not keep walking past the destination, which is exactly what naive looping does.

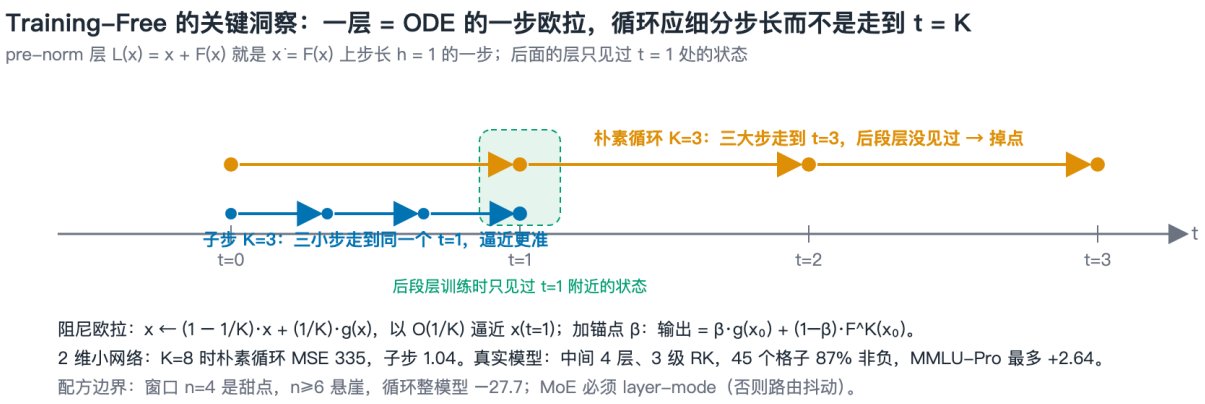


Figure (labels in Chinese): naive looping pushes the state to t=3, a region later layers never saw; sub-stepping reaches the same t=1 more accurately.

One sentence

Without touching a single weight or adding a parameter, treat four contiguous mid-stack layers of a released model as the vector field of an ODE and re-integrate it to the *same* endpoint with K damped Euler (Runge–Kutta-style) sub-steps at inference; across seven model families (Qwen3, Llama-3.2, Qwen1.5-MoE, Moonlight, DeepSeek-V2-Lite) and 45 (model, benchmark) cells, 87% are non-negative and knowledge-heavy multiple-choice benchmarks such as MMLU-Pro and GPQA gain about 2 points.

1. The problem

Every earlier looped transformer had to be *trained* to loop: the parameters are tied and optimized under the loop structure, so loop and weights are inseparable. Almost every publicly released checkpoint — each the endpoint of continued pretraining, SFT, and several rounds of RLHF/DPO — therefore cannot benefit from looping.

The paper asks: **can a frozen, off-the-shelf checkpoint be looped directly at inference, with no fine-tuning, no continued training, no auxiliary parameters, and no architectural change?**

Three existing results point to yes: Men et al. (2025) delete whole mid-layer blocks with minimal loss; Lad et al. (2024) skip, swap, or repeat mid-layers without catastrophe; Belrose et al.'s (2023) tuned lens shows intermediate logits already encode much of the final prediction. The standard reading is "mid-layers are redundant, so we can remove them". The authors add the other half: **the same redundancy that makes a mid-layer safe to delete makes it safe to re-apply.**

2. Method

2.1 The wrapper

Let $f = L_{N-1} \circ \cdots \circ L_0$; choose a contiguous window $[a, b]$ and set $g = L_b \circ \cdots \circ L_a$. The wrapped model is

$$\hat{f}(x) = \underbrace{(L_{N-1} \circ \cdots \circ L_{b+1})}_{\text{post-loop}} \circ g^{(K)} \circ \underbrace{(L_{a-1} \circ \cdots \circ L_0)}_{\text{pre-loop}}(x)$$

at an extra cost of $(b - a + 1)(K - 1)$ layer forwards. $g^{(K)}$ comes in two modes:

- **block-mode**: $(L_b \circ \cdots \circ L_a)^K$ — the whole window repeated K times.
- **layer-mode**: $L_b^K \circ \cdots \circ L_a^K$ — each layer repeated K times before moving on.

For dense models the two are similar. **MoE models require layer-mode**: in block-mode every iteration shows the gating network a slightly perturbed state, routing to different experts, and the accumulated routing noise overwhelms the intended refinement ("routing thrash"). Layer-mode computes the gate once and applies the same expert mixture K times.

2.2 The key insight: one layer = one forward-Euler step

A pre-norm layer is $L(x) = x + F(x)$ with F the attention + MLP residual. Define the window residual field $F_g(x) := g(x) - x$; then $g(x) = x + 1 \cdot F_g(x)$ is exactly one forward-Euler step of size $h = 1$ on the autonomous ODE

$$\dot{x} = F_g(x).$$

The post-loop layers were trained on this approximation of $x(t{=}1)$.

Why naive looping fails: applying $x \leftarrow g(x)$ K times is K Euler steps of size 1, landing at $x(t{=}K)$, a point on the trajectory the post-loop layers have never seen. A toy network with a 2-D bottleneck (Figure 2) shows the naive endpoints drifting out of the trained low-loss valley as K grows: MSE goes from 0.015 to 2.88 at $K = 2$ and to 335 at $K = 8$.

The right target is not $t = K$ but a finer integration of $[0, 1]$. K Euler steps of size $h = 1/K$ give the damped update

$$x_{k+1} = \left(1 - \tfrac{1}{K}\right) x_k + \tfrac{1}{K}\, g(x_k),$$

which converges to the true $x(t{=}1)$ at order $O(1/K)$ and strictly improves on the single step the network already implements. On the toy problem, sub-stepping at $K = 8$ gives MSE 1.04 versus 335 for naive looping — a ~320× gap.

2.3 A particularly effective Runge–Kutta tableau

Generalize damped Euler to s-stage explicit RK, $x_1 = x_0 + h \sum_i b_i k_i$. The authors pick $h = 1$, $s = K$, $a_{ij} = \frac{1}{K}\,\mathbb{I}(j < i)$, and output weights

$$b_1 = \beta + \frac{1 - \beta}{K}, \qquad b_i = \frac{1 - \beta}{K} \;(i \geq 2),$$

under which the RK output is exactly

$$x_1 = \beta\, g(x_0) + (1 - \beta)\, F^K(x_0),$$

an **interpolation between the original single-step output and the K-step damped-Euler endpoint** (Algorithm 3). $\beta = 0$ is pure sub-stepping, $\beta = 1$ leaves the model untouched; intermediate β biases the trajectory toward the trained one-step endpoint. This anchor weight is the method's safety valve.

Seven iteration strategies are compared (Table 1): naive loop, forward Euler, midpoint/Heun (RK2), RK4, heavy-ball, Anderson acceleration, Aitken Δ^2, and Uniform Loop. **No fixed-point accelerator robustly beats K-stage RK**, which the authors read as evidence that the looped middle of a pretrained transformer "is not contractive in any useful sense" — it is a segment of an ODE flow, not a fixed-point iteration.

3. Key experiments and numbers

One recipe, **no per-cell tuning**: 3-stage RK on the middle 4 layers, block-mode for dense, layer-mode for MoE; lm-eval-harness 0.4.11, bf16; ~20,000 H100 hours.

3.1 Main results (Table 3)

Model	Benchmark	Base	Looped	Δ pp
Qwen3-4B-Instruct	MMLU-Pro 5-shot	0.5714	0.5979	**+2.64**
	GPQA-Main 0-shot	0.3371	0.3571	**+2.01**
	CommonsenseQA 7-shot	0.7887	0.7993	+1.06
Llama-3.2-3B-Instruct	GPQA-Main	0.2991	0.3103	+1.12
	MMLU-Pro	0.3164	0.3236	+0.71
Llama-3.2-1B-Instruct	GPQA-Main	0.2790	0.2969	+1.79
Qwen1.5-MoE-A2.7B	ARC-Challenge 25-shot	0.4829	0.5060	**+2.30**
	CommonsenseQA	0.7961	0.8133	+1.72
	OpenBookQA	0.3160	0.3320	+1.60
Moonlight-16B-A3B	MMLU 0-shot	0.6786	0.6860	+0.74
	OpenBookQA	0.3160	0.3280	+1.20
DeepSeek-V2-Lite-Chat	ARC-Challenge	0.5794	0.5879	+0.85

The largest gains (+2.0 to +2.6) are on the hardest knowledge MC tasks (MMLU-Pro, GPQA, ARC-C) with the strongest MHA backbones; MLA+MoE models (DeepSeek-V2-Lite, Moonlight) move the same way at 3–4× smaller magnitude.

3.2 Window size: $n = 4$ is the sweet spot, $n \geq 6$ a cliff (Table 2a)

Qwen3-1.7B-Base, forward Euler $K = 2$, 16-task average:

Window layers n	1	2	3	**4**	6	12	28 (all)
Δ	+0.18	+0.06	+0.21	**+0.55**	−0.82	−0.63	−27.73

$n = 4$ wins on Qwen3-1.7B/4B and Llama-3.2-3B ($n = 3$ slightly better on 0.6B). Looping the whole model loses 27.7 points — a sharp contrast with the "model-loop" paradigm on *trained* looped models. **An untrained model can only be looped in a small mid-stack window.**

3.3 The depth-fraction rule (§3.2)

Across eight architectures the optimal window center lies in a narrow band of fractional depth: 0.43–0.71 (mode ≈ 0.50) for models above 1.7B, shifting earlier to 0.25–0.56 below 1B. Hypothesis: head specialization concentrates in late layers, which the loop must avoid; in small and heavily distilled models the "late" fraction is larger, so the safe window starts earlier.

3.4 Leakage-free transfer: Qwen3-30B-A3B (Table 4)

Configuration $[22, 24]$ fixed in advance from the depth-fraction rule, committed before any cell was scored. 8 of 9 benchmarks improve: CSQA +1.14, MMLU-flex +0.79, SuperGPQA +0.70, TruthfulQA +0.49; LAMBADA perplexity $4.12 \rightarrow 4.11$. All positive but small — the rule transfers, and gains on a large MoE are modest.

3.5 The full picture (§3.5)

45 cells: 27 positive ($\Delta > +0.3$, 60%), 12 neutral (27%), 6 negative. Negatives concentrate in **sub-3B distilled checkpoints on knowledge MC** (e.g. Llama-3.2-1B on MMLU). Even there, GPQA flips positive on Llama-3.2-1B (+1.79) with a very early window $[4, 7]$ — the failure boundary is task-dependent, not absolute.

3.6 Against other looping methods (§3.7, Figures 4–5)

- Naive looping à la Giannou et al. (2023) collapses on **every** cell; at $K = 6$ the 16-task mean falls to 37.89% (−17.71 pp).

- Uniform Loop (Lys et al. 2026) fails for $K \geq 6$.
- This method stays stable for $K \in \{1, \ldots, 24\}$.

4. Why it matters

1. **It lowers the barrier.** Looping used to mean "train from scratch, big-lab only"; now anyone gets 1–2.5 points on an existing open model for free.
2. **It supplies a testable theory.** "Layer = Euler step" is not new (Neural ODE, DEQ), but turning it into an inference-time prescription — keep the endpoint, refine the step, anchor with β — is. The same view explains why naive looping fails, why MoE needs layer-mode, and why fixed-point accelerators do not help.
3. **Layer-mode for MoE** agrees with Loopie's (2607.16051) independently derived "layer-loop scales better on MoE"; one paper trains nothing, the other trains from scratch, and both land on the same conclusion.

5. Limitations

- Gains cap at ~2–3 points, an order of magnitude below from-scratch looped models (Huginn: +20 on ARC-E). This squeezes existing weights; it is not a new scaling axis.
- Gains concentrate on knowledge MC; the paper reports no systematic improvement on generative tasks (GSM8K, code).
- Extra inference cost of about $4 \times (K - 1)$ layer forwards ($K = 3 \approx 8$ extra layers), i.e. 15–30% latency on 28–48-layer models.
- Small distilled models lose points and need family-specific window placement; "no tuning" is per-cell — the recipe itself was found on these models.

6. Relation to other work

- Theory: DEQ (Bai et al. 2019, 2020) treats weight-tied nets as fixed-point solvers; this paper argues the pretrained mid-stack behaves as an ODE flow, not a contraction.
- Training-free depth manipulation cousins: layer skipping (Men 2025, Lad 2024); Program-of-Layers (Li et al. 2026, ICML) learns a small network that decides per input which layers to skip or repeat — a learnable version of this idea.
- Retrofit-with-training: Relaxed Recursive Transformers (Bae 2025, per-layer LoRA), Retrofitted Recurrence (McLeish 2025, curriculum continued training) — they train and gain more, forming a zero-cost / low-cost / from-scratch spectrum with this paper.

7. Practical takeaways

- To try looping on an existing model: middle 4 layers, 3-stage RK, intermediate β, block-mode for dense, layer-mode for MoE. Validate on knowledge MC first; the signal is clearest there.
- Never repeat layers naively. Any "run this block twice" experiment should use the damped update $x \leftarrow (1 - 1/K)x + (1/K)g(x)$ instead of $x \leftarrow g(x)$.
- On MoE, freeze routing before repeating anything.
- Most valuable when a model is already strong and stuck on the last few points; for small models or generative reasoning, look at from-scratch approaches.

Appendix: quick numbers

- Recipe: middle 4 layers, $K = 3$ RK, dense block / MoE layer.
- 45 cells: 60% positive, 27% neutral, 87% non-negative; ~20k H100 hours.
- Qwen3-4B-Instruct MMLU-Pro +2.64, GPQA +2.01; Qwen1.5-MoE ARC-C +2.30.
- Window $n = 4$ +0.55, $n = 6$ −0.82, whole model −27.73; naive loop $K = 6$ −17.71.
- Optimal window fractional depth: 0.43–0.71 (>1.7B), 0.25–0.56 (<1B).

Glossary

Term	One-line meaning
Recurrent depth / looped transformer	The same parameters are applied repeatedly along depth; effective depth = stored layers × loop count, parameters unchanged
model-loop / block-loop / layer-loop	Loop the whole stack / only a middle segment / each layer several times before the next; see `assets/figures/diagrams/loop_granularity.png`
Input injection	Re-feed the input embedding at every loop; without it the iteration is unstable

Term	One-line meaning
Truncated backprop	Compute gradients through only the last k loops so memory is independent of the loop count
Adaptive halting (ACT)	Let each token decide how many loops it needs
Path independence	Different random initial states converge to the same behavior
KV cache	Cached attention keys/values; looped models add one copy per loop, the main memory bottleneck
MoE	Mixture of experts: each token activates a small subset of parameters; “20B-A2B” = 20B total, 2B active
FLOP-matched vs measured-compute-matched	Aligning theoretical compute vs aligning measured training step time — the activation memory looping saves only shows up in the latter
Latent reasoning vs explicit CoT	Extra computation in hidden space vs intermediate steps written as text tokens
Fixed point	A state that another loop leaves unchanged; one line of work trains loops to converge to fixed points for stability
PLT / CLP	Parallel Loop Transformer / cross-loop position offset — shift the previous round’s states by one token so rounds run in parallel

Report 04 · LoopCoder-v2: In a Parallel Loop Transformer, the Second Loop Is Almost Everything

Item	Detail
Paper	LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling
arXiv	2606.18023 (Jun 2026)
Authors	Jian Yang, Shawn Guo, Wei Zhang, Tianyu Zheng, Yaxin Du, Haau-Sing Li, Jiajun Wu, Yue Song, Yan Xing, Qingsong Cai, Zelong Huang, Chuan Hao, Ran Tao, Xianglong Liu, Wayne Xin Zhao, Mingjie Tang, Weifeng Lv, Ming Zhou, Bryan Dai (Beihang U. / IQuest Research / Langboat / Renmin U.)
Model	Multilingual-Multimodal-NLP/LoopCoder-V2
In this repo	Original `papers/original/2606.18023.pdf` · Chinese `papers/zh/2606.18023.zh.pdf`

At a glance

Problem · The Parallel Loop Transformer flattens the latency and memory of looping — so how many loops should be used at deployment, and why?
Method · Train four 7B coders with R=1/2/3/4 from scratch on 18T tokens under one recipe, same SFT, then diagnose hidden states, attention and output shift per loop.
Result · R=2 lifts SWE-bench Verified from 43.0 to 64.4; R=3 falls to 27.6, R=4 to 22.4. Effective rank peaks at loop 2; the fixed offset cost is 30–45× the marginal gain from loop 3 on.
Limits · PLT-specific (sequential looping has no such tax); one run per R, single seed; no ablation without the offset.
Plain words · One extra round of thinking pays; but this parallel kind of thinking charges a fixed tax per round, and from the third round the tax exceeds the gain.

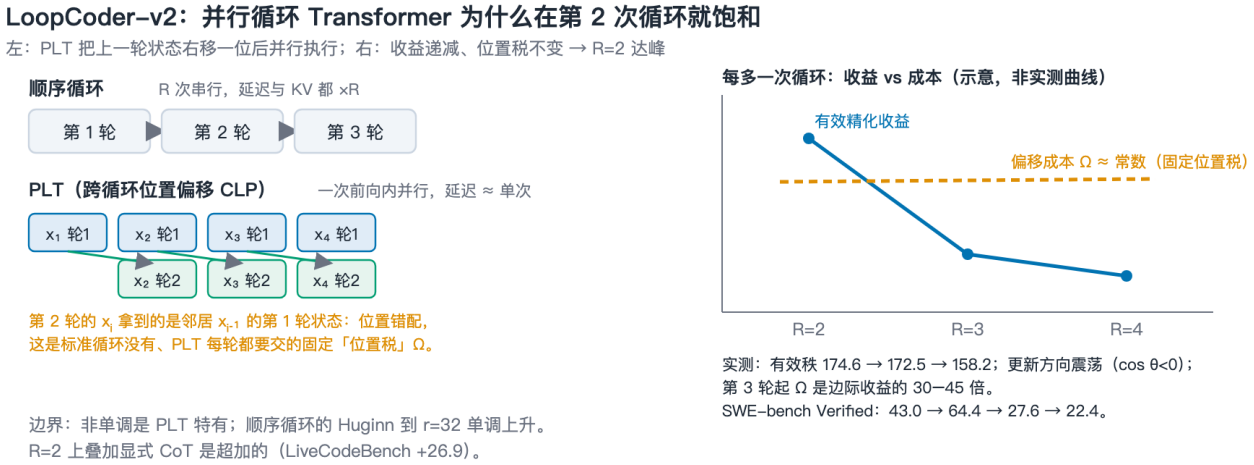


Figure (labels in Chinese): sequential looping vs PLT parallel execution (left) and the diminishing-gain / constant-cost picture (right).

One sentence

Train four 7B Parallel Loop Transformer (PLT) coders with loop counts $R \in \{1, 2, 3, 4\}$ from scratch on 18T tokens under one recipe, then instruction-tune them identically: $R = 2$ lifts SWE-bench Verified from 43.0 to 64.4 and Multi-SWE from 14.0 to 31.0, while $R = 3$ falls to 27.6 and $R = 4$ to 22.4. Per-loop diagnostics show that the second loop carries nearly all productive refinement; after it, marginal gains collapse while the cross-loop position offset (CLP) that PLT introduces for parallelism charges a fixed "positional tax" 30–45× larger than the gain from loop 3 onward.

2. Background: why PLT

Standard sequential looping has a deployment-killing property: R loops mean R serial passes through the shared block, so latency scales by R; each layer caches KV at every loop, so memory is $O(R \cdot L \cdot S \cdot d)$, also scaling by R.

The Parallel Loop Transformer (PLT; Wu et al. 2025, arXiv 2510.24824) flattens both to near-constant with two mechanisms (Table 1):

	Sequential loop	PLT
Execution	serial	parallel, single pass
Latency	$O(R\,C_{\text{block}})$	$\approx C_{\text{block}}$
KV cache	$O(RLSd)$	$O(LSd)$
Inter-loop input	$\mathbf{h}^{(r-1)}$	$\text{Embed}(x) + \text{shift}(\mathbf{h}^{(r-1)})$

1. **Shared KV + gated sliding-window attention (G-SWA).** The first loop's KV $K_{\text{share}}, V_{\text{share}}$ is frozen and shared with all later loops; each later attention layer outputs a per-head gated mix of a global branch (full attention over the frozen KV) and a local branch (sliding-window attention of width $w = 64$ over the current loop's KV): $\tilde{y}^{(r)} = g \odot y^{(r)}_{\text{global}} + (1-g) \odot y^{(r)}_{\text{local}}$, $g = \sigma(f_{\text{gate}}(\text{RMSNorm}(\mathbf{h})))$.
2. **Cross-loop position offset (CLP).** The input to loop $r \geq 2$ is $B^{(r)} = \text{Embed}(x) + \text{shift}(\mathbf{h}^{(r-1)})$, where shift moves the previous loop's hidden states **one position to the right**. Loop r of token x_i can then run in the same forward pass as loop $r+1$ of token x_{i-1}.

The price: at loop r, token x_i receives not its own previous state but its neighbor x_{i-1}'s. This is the **positional mismatch** — an information constraint unique to PLT that standard looping never incurs.

3. Question and method

PLT makes loop count a free deployment choice, which raises the question: how many? Training every candidate is expensive and explains nothing. The authors frame it as gain versus cost: each extra loop offers marginal refinement on one side and CLP's fixed mismatch cost on the other.

3.1 Macroscopic protocol

- 7B dense PLT, G-SWA window 64, first-loop KV sharing.
- Pretrained from scratch on 18T tokens, text:code 1:1, 100+ programming languages; Adam (0.9, 0.95), $\epsilon = 10^{-15}$, weight decay 0.1, LR 4×10^{-4} cosine with 5% warm-up; bf16; **1M GPU hours** total.
- Training and inference loop counts matched (trained at $R = r$, evaluated at $R = r$).
- All four models instruction-tuned on the same 6M examples; final checkpoint reported.
- Infrastructure: custom Megatron-LM unrolling R loops into $R \cdot L$ scheduled layers with a single set of parameters; all loop instances of a layer co-located on one pipeline stage so weight sharing is communication-free.

3.2 Microscopic diagnostics (§3.2)

Four per-loop lens groups; a mechanism is credited only when they agree:

- **Hidden-state dynamics**: step size $\delta^{(r)} = \|\mathbf{h}^{(r)} - \mathbf{h}^{(r-1)}\|$; cosine between successive updates $\cos\theta^{(r)}$ (<0 = direction reversal, oscillation); effective rank erank (entropy-exponential of normalized singular values, a scale-free diversity measure); fixed-point gap $\Delta^{(r)}_{\text{FP}} = \|\mathbf{h}^{(r)} - f_\theta(\mathbf{h}^{(r)})\|$.
- **Attention evolution**: attention entropy, inter-loop attention KL $D^{(r)}_{\text{KL}}$, head–head cosine similarity, mean G-SWA gate $\bar{g}^{(r)}$.
- **Output-distribution shift**: logit-lens rank of the gold token, inter-loop output KL $\Delta p^{(r)}$, output entropy.
- **Intrinsic offset cost** $\Omega^{(r)} = \frac{1}{S}\sum_i \|\mathbf{h}_i^{(r-1)} - \mathbf{h}_{i-1}^{(r-1)}\|$: mean distance between adjacent token representations. The less alike neighbors are, the more distortion from substituting a neighbor's state for one's own.

4. Key results

4.1 Main table (Table 2, four 7B variants)

Model	HE+	MultiPL-E	BCB	LCB	SWE-V	Multi-SWE	TB-v1	TB-v2	M2W	BFCL	Avg
Baseline $R = 1$	81.1	69.5	40.1	27.4	43.0	14.0	26.3	11.2	35.3	32.2	38.0
$R = 2$	**84.1**	**73.9**	**46.1**	**35.4**	**64.4**	**31.0**	**34.2**	**21.0**	34.5	**40.1**	**46.5**

Model	HE+	MultiPL-E	BCB	LCB	SWE-V	Multi-SWE	TB-v1	TB-v2	M2W	BFCL	Avg
$R = 3$	75.0	69.8	43.3	28.6	27.6	11.0	30.0	12.2	35.1	36.3	36.9
$R = 4$	76.8	67.3	40.8	24.5	22.4	9.3	26.3	9.0	**41.4**	39.5	34.3

Three observations:

- **Strongly non-monotonic.** $R = 2$ averages +8.5; $R = 3$ is 1.1 *below* baseline; $R = 4$ lower still. SWE-bench Verified: 43.0 → 64.4 → 27.6 → 22.4.
- **One extra loop puts a 7B model among much larger systems.** 64.4 on SWE-bench Verified beats Kimi-Dev-72B (60.4) and Qwen3-235B-A22B (45.2) and approaches Qwen3-Coder-480B (67.0). On the held-out agentic SWE-bench-CC it reaches 33.4%.
- **Gains concentrate on agentic and reasoning tasks.** SWE, Terminal-Bench, LiveCodeBench move most; Mind2Web is best at $R = 4$ — single-benchmark noise the authors do not over-read.

4.2 Diagnostics: what loop 2 does and why loop 3 hurts (§4.2, Table 3)

PLT4 statistics over 500 held-out samples:

Loop	Step δ	Output shift Δp	Eff. rank	$\cos\theta$
$r = 2$	**846**	**1.75**	**174.6**	−0.72
$r = 3$	464	1.32	172.5	−0.46
$r = 4$	1014	1.58	158.2	0.04

- **Loop 2 is the principal site of productive refinement**: largest attention re-routing $D_{\mathrm{KL}}^{(2)}$, largest output shift, peak effective rank.
- **After loop 2, returns collapse**: Δp and D_{KL} drop sharply; effective rank declines from its peak — later loops **narrow** the representational subspace rather than enrich it; attention heads grow redundant (Figure 4); update directions reverse ($\cos\theta < 0$) — oscillation, not convergence. In the four-loop model, loop 3 is nearly a dead pass-through and loop 4's large step is output readout, not new refinement (it has the lowest effective rank).
- **The gate stays above 0.5**: later loops keep leaning on the frozen loop-1 global KV rather than constructing qualitatively new context.
- **The offset is a fixed tax**: $\Omega^{(r)}$ is roughly constant across loops while $\Delta p^{(r)}$ collapses after loop 2. Figure 3's "scissors": at every extra loop the offset cost exceeds the marginal gain by **30–45×**.

Diminishing gains against a fixed cost: the net effect peaks at $R = 2$ and turns negative afterward.

4.3 Explicit CoT and latent looping are complementary (Table 4)

At the optimal $R = 2$, the instruction model (latent loop only) versus the thinking model (explicit CoT + loop):

$R = 2$	LCB	CRUX	MultiPL-E	FullStackBench	BCB-Hard
Instruct (loop only)	35.4	86.9	73.9	47.2	23.7
Thinking (CoT + loop)	**62.3**	**93.5**	**77.8**	**49.9**	**26.4**
Δ	+26.9	+6.6	+3.9	+2.7	+2.7

LiveCodeBench +26.9 far exceeds the sum of the two ingredients: explicit CoT alone does not improve the non-looped model on these tasks, and the loop alone gives single digits. The authors' explanation is that the two act at different granularities — CoT decomposes the problem into textual steps, the loop refines the representation under each step — so they compound **super-additively**. This is the paper's most forward-looking conclusion: **latent depth recurrence and explicit reasoning are complementary axes of test-time compute, not substitutes.**

5. Where the conclusion applies (important)

"Two loops suffice, three regress" tends to lose its preconditions in circulation. The paper's own boundaries are clear:

1. **It is a PLT-specific phenomenon.** The mechanistic explanation of non-monotonicity depends on the fixed positional tax of CLP; standard sequential looping has no such term. Huginn (sequential, input-injected) improves monotonically

from $r = 4$ to $r = 32$; Ouro uses 4 steps; Fixed-Point Reasoners and STARS study how to keep dozens of loops stable.
2. **Training and inference loop counts are tied.** Each R is a separately trained model, not one model evaluated at different R. Here R is part of the training recipe, not a test-time compute knob.
3. **Same $R = 2$ answer as Loopie, different reason.** Loopie picks 2 because marginal returns decay fastest under a **fixed pretraining compute budget**; LoopCoder-v2 picks 2 because of CLP's fixed cost. The two papers share authors (Bryan Dai and Ran Tao appear on both), so they can be read as one group answering the same question from a compute-matching angle and a mechanistic angle.
4. **Effective rank is an actionable diagnostic.** If effective rank is still rising at the candidate loop, another loop may still refine; once it turns down, further loops mostly pay the positional tax. No exhaustive training sweep needed.

6. Limitations

- One run per variant, single seed; a 1M-GPU-hour budget rules out replication. The $R = 3$ drop on SWE (−15.4) is too large to be noise, but the reverse swing on Mind2Web is a reminder that the main table has variance.
- One architecture (PLT), one scale (7B dense). Whether "two is optimal" changes with scale or MoE backbones is unknown (SMELT, 2609.01343, independently lands on "loop the middle half twice" for MoE).
- No ablation removing CLP (what would sequential looping do at $R = 3$?). The mechanistic account is therefore convergent evidence, not a causal experiment.
- The 18T pretraining mixture is not released; only the code-language distribution is given.

7. Relation to other work

- Predecessor: PLT (Wu et al. 2025) supplies the architecture; LoopCoder-v1 details are not expanded here.
- Concurrent work on loop count: Schwethelm et al.'s (2026) iso-depth scaling law gives $\varphi = 0.46$ — looping r times is worth only $r^{0.46}$ unique blocks; Loopie's $N\times$ loop vs. $N\times$ layer comparison; Yang et al. (2026, STARS) observe peak-then-collapse with depth and fix it with Jacobian spectral-radius regularization.
- Efficiency line: MELT (cross-loop shared KV with gated updates), Looped Latent Attention (low-rank cross-loop KV compression), LT2 (linear attention + loops). PLT is the representative that also flattens latency.
- The CoT-complements-looping view echoes Xu & Sato's (2025) formal comparison: loops excel at deterministic parallel computation (DAG evaluation), CoT at approximate inference over self-reducible problems.

8. Practical takeaways

- Deploying PLT-style parallel loops: $R = 2$ is the default — one extra forward buys ~20 points on SWE-style tasks.
- To decide whether one more loop helps, look at the effective-rank curve rather than sweeping training runs.
- "Two loops is enough" is not a universal ceiling for recurrent depth; it is the price of parallelization. If the latency budget allows sequential looping, the Huginn/Ouro line goes much deeper.
- Treat the latent loop as the substrate under CoT, not a replacement: loop first, then add a thinking mode, and the gains compound.

Appendix: quick numbers

- 7B PLT, 18T tokens, 1:1 text/code, 1M GPU hours; $R \in \{1, 2, 3, 4\}$ each trained from scratch.
- SWE-bench Verified: 43.0 → **64.4** → 27.6 → 22.4; Multi-SWE: 14.0 → **31.0** → 11.0 → 9.3; average 38.0 → **46.5** → 36.9 → 34.3.
- PLT4 effective rank: 174.6 / 172.5 / 158.2 ($r = 2/3/4$); $\cos\theta$: −0.72 / −0.46 / 0.04.
- Offset cost $\Omega \approx$ constant, 30–45× the marginal gain from loop 3 onward.
- Thinking vs. Instruct at $R = 2$: LCB +26.9, CRUX +6.6.

Glossary

Term	One-line meaning
Recurrent depth / looped transformer	The same parameters are applied repeatedly along depth; effective depth = stored layers × loop count, parameters unchanged
model-loop / block-loop / layer-loop	Loop the whole stack / only a middle segment / each layer several times before the next; see `assets/figures/diagrams/loop_granularity.png`
Input injection	Re-feed the input embedding at every loop; without it the iteration is unstable

Term	One-line meaning
Truncated backprop	Compute gradients through only the last k loops so memory is independent of the loop count
Adaptive halting (ACT)	Let each token decide how many loops it needs
Path independence	Different random initial states converge to the same behavior
KV cache	Cached attention keys/values; looped models add one copy per loop, the main memory bottleneck
MoE	Mixture of experts: each token activates a small subset of parameters; “20B-A2B” = 20B total, 2B active
FLOP-matched vs measured-compute-matched	Aligning theoretical compute vs aligning measured training step time — the activation memory looping saves only shows up in the latter
Latent reasoning vs explicit CoT	Extra computation in hidden space vs intermediate steps written as text tokens
Fixed point	A state that another loop leaves unchanged; one line of work trains loops to converge to fixed points for stability
PLT / CLP	Parallel Loop Transformer / cross-loop position offset — shift the previous round’s states by one token so rounds run in parallel

Report 05 · Loop the Loopies!: The First Compute-Matched Win for Looped MoE

Item	Detail
Paper	Loop the Loopies!
arXiv	2607.16051 (Jul 2026)
Authors	Zitian Gao, Yilong Chen, Yihao Xiao, Xinyu Yang, Ran Tao, Joey Zhou, Bryan Dai (IQuest Research)
Models	Loopie-20B-A2B, Loopie-6B-A0.6B (Base / Thinking)
In this repo	Original `papers/original/2607.16051.pdf` · Chinese `papers/zh/2607.16051.zh.pdf`

At a glance

Problem · Given the same pretraining compute, can a looped transformer beat a vanilla MoE that simply has more parameters?
Method · Loop each layer twice, halve stored depth → activation memory halves → double the per-device micro-batch → spend the speed-up on width; match by measured step time; SPT + RL post-training.
Result · At equal budget over 800B tokens it overtakes a reproduced Qwen3-30B-A3B after ~600B; the 20B-A2B Thinking model reaches MMLU 81.28 and AIME24 92.09 with one-seventh of the peers' tokens.
Limits · Matched on training step time, not FLOPs (nominal +42%); inference cost unreported; the reference is an 800B-token reproduction.
Plain words · The win does not come from loops replacing parameters; it comes from the memory loops save, spent on a wider model that wins within the same training time.

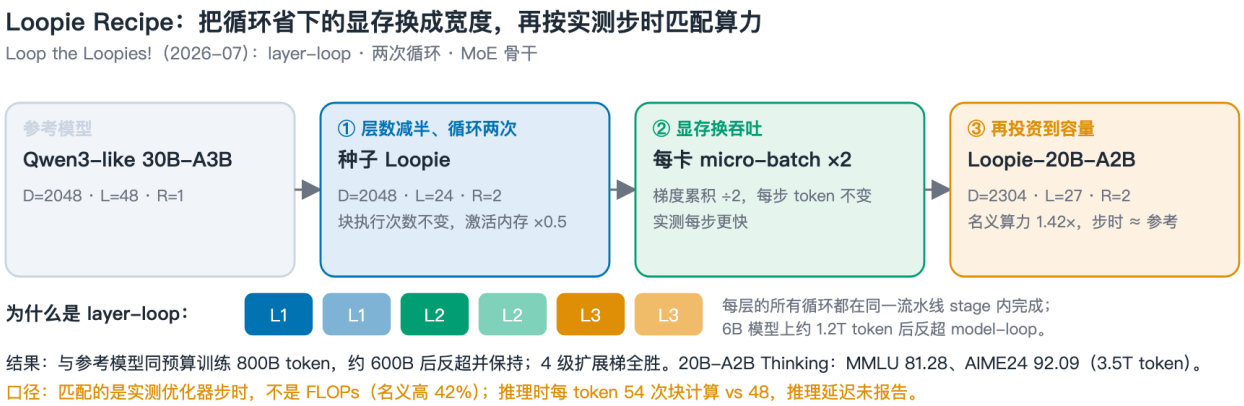


Figure (labels in Chinese): the three-step Loopie Recipe and the layer-loop execution order.

One sentence

Looped transformers have lived under one objection: given $N\times$ the pretraining compute, scaling parameters by N usually beats looping N times. Loopie answers with three moves — (1) replace whole-model looping with per-layer looping (layer-loop), (2) match compute by **measured optimizer-step wall-clock time** rather than theoretical FLOPs (the Loopie Recipe), (3) loop only twice. Loopie-20B-A2B, trained for 800B tokens at **the same per-step wall-clock time** as a reproduced Qwen3-30B-A3B, overtakes it after ~600B tokens and stays ahead; with 3.5T pretraining tokens, 2T tokens of "supervised pre-training" (SPT), and math/code RL, the 20B-A2B Thinking model reaches MMLU 81.28 and AIME24 92.09, matching or beating Nemotron 3 Nano (25T tokens) with less than one-seventh of its pretraining tokens.

1. The problem

The introduction's central question: **can looped transformers match or exceed vanilla transformers under the same pretraining compute budget?**

Earlier looped work (Ouro, Huginn) treated looping mainly as a **parameter-efficiency** device: stronger at equal parameter count. But looping N times multiplies training compute by roughly N. To make looping a **compute-efficiency** device, it must beat a vanilla model that spends the same compute on more unique parameters — and under that comparison looped models had historically lost (Csordás 2024, Frey 2026). Since frontier models are now almost all MoE (Qwen3, Kimi K2.5,

GLM-5, DeepSeek-V4, MiniMax-M2), a looping scheme that only works on dense backbones has no path to the frontier.

2. Method

2.1 Layer-loop: a different loop order

Three layers, two loop steps:

- **model-loop** (Ouro, Huginn): Layer1 → Layer2 → Layer3 → Layer1 → Layer2 → Layer3
- **layer-loop** (Loopie): Layer1 → Layer1 → Layer2 → Layer2 → Layer3 → Layer3

Each layer iterates locally on the hidden state, then hands the result to the next layer. Three reasons:

1. **Better measured scaling.** On Loopie-6B-A0.6B, layer-loop trails model-loop early and overtakes it after ~1.2T tokens, pulling away thereafter (Figure 2).
2. **Infrastructure-friendly.** Repeated applications of one layer are adjacent in the forward and backward graphs; parameter and gradient reuse distances are short, activation checkpointing and gradient accumulation are simpler. Under pipeline parallelism all loops of a layer stay on one stage; model-loop must route the last stage's output back to the first, creating cyclic dependencies, extra communication, and pipeline bubbles.
3. **A more natural sharing pattern.** A 48-layer model with 2 model-loop steps asks physical layer 3 to serve effective depths 3 and 51; transformer representations are known to be non-uniform across depth (early and late layers differ from the homogeneous middle). Layer-loop reuses a layer only at adjacent effective depths — more local sharing.

Ablation (§2.7, Figure 5): same backbone, data, tokens, and compute budget, but with layer-loop removed, the 6B-A0.6B model is clearly worse. The improvement cannot be attributed to extra computation; the schedule of recurrence matters.

2.2 The Loopie Recipe: match compute by measured step time

Leading-order pretraining compute $\mathcal{C}(D, L, R) \propto LD^2R$ (width D, stored layers L, loops R); dominant activation memory $M_{\text{act}} \propto s\,b\,D\,L$ — it scales with **stored** depth, not executed depth, because all loops of a layer sit inside one checkpointing unit. Three steps:

1. From the non-recurrent reference (Qwen3-like 30B-A3B: $D_0 = 2048$, $L_0 = 48$, $R_0 = 1$), **halve the layers and loop twice** to get a seed (2048, 24, 2). Block executions $LR = 48$ unchanged, leading compute unchanged, activation memory halved.
2. Spend the freed memory on **doubling the per-device micro-batch** ($b_1 = 2b_0$, gradient-accumulation steps halved, tokens per step unchanged) and measure the training-efficiency gain S_{mb}.
3. **Reinvest** the gain into capacity: sweep (D, L) around the seed, keep candidates that still support 2× micro-batch, and select the one whose **measured optimizer-step time** is closest to the reference.

Configuration	D	L	R	Norm. compute $\widehat{C}$	Norm. act. memory
Qwen3 30B-A3B (reference)	2048	48	1	1.00×	1.00×
Seed Loopie	2048	24	2	1.00×	0.50×
Candidate 1	2176	25	2	1.18×	0.55×
Candidate 2 → Loopie-20B-A2B	**2304**	**27**	**2**	**1.42×**	**0.63×**
Candidate 3	2432	28	2	1.65×	0.69×

Loopie-20B-A2B nominally does 42% more transformer-block work, yet in Megatron-LM (EP=8, MBS=2 vs. the reference's EP=8, MBS=1) its per-step wall-clock matches the reference. **The authors state explicitly that this is not FLOP equality but measured-cost equality** — the entire efficiency gain comes from the doubled micro-batch.

2.3 Why only two loops (§2.8)

At fixed stored parameters and step count, more loops almost always improve the training curve — but that is not compute-matched. Under a fixed budget, more loops mean fewer tokens, a smaller model, or a stronger vanilla baseline. Comparing $N\times$ layer-loop against $N\times$ stored layers at 250B tokens (Figure 6): **the marginal benefit of recurrence is largest at $R = 2$ and decays quickly.**

The authors add a caveat: this does not claim large loop counts are ineffective — cheap inference compute, adaptive

computation, or studying recurrent reasoning per se all justify large R. But **frontier pretraining is bounded by total compute, and there $R = 2$ is the best trade-off between iterative refinement, throughput, and compute-matched scaling.**

3. Key results

3.1 Compute-matched comparison (§2.5, Figure 3)

Loopie-20B-A2B versus a self-reproduced Qwen3-30B-A3B, each trained for 800B tokens under **the same pretraining budget**. Loopie trails early and **overtakes after ~600B tokens**, holding the lead (average over eight benchmarks: ARC-C/E, BoolQ, CSQA, HellaSwag, MMLU, OBQA, WinoGrande).

3.2 Scaling ladder (§2.6, Figure 4)

Four rungs: non-recurrent baselines at 0.15B / 0.25B / 0.5B / 1B active parameters versus compute-matched Loopies at 0.10B / … / 0.70B, each trained on 1000× its active parameter count in tokens (150B / 250B / 500B / 500B, the last capped by compute). **Loopie leads on all four rungs, and the gap does not shrink with scale.**

3.3 Pretraining recipe (§3)

- Stage 1: Nemotron-CC-v2-HQ, ~570B unique tokens, **4 epochs** (2.28T) — repeating high-quality data beats one pass over more, lower-quality data.
- Stage 2: 1.26T tokens of high-quality annealing data (Nemotron SFT-v1 27.8%, Specialized-v1 21.9%, Code-v2 20.7%, CC-v2-HQ-Synthetic 15.6%, CC-Math-v1 10.0%, …).
- ~3.5T total. Qwen3 tokenizer; AdamW, $\epsilon = 10^{-15}$, weight decay 0.1, LR 5×10^{-4} (6B) / 3×10^{-4} (20B), **warm-up-stable with no decay**; batch 1024 × 8192.
- Initialization: embeddings $\mathcal{N}(0, d^{-1})$, block parameters SmallInit $\mathcal{N}(0, \frac{2}{5d})$ — the same Takase-style scheme Huginn used.

3.4 Post-training: Supervised Pre-training (SPT) + RL (§4)

SPT is the paper's second methodological contribution: SFT's loss masking (loss on target tokens only) combined with PT's optimization scale (global batch ≥1024, sequence ≥128K, ≥128M token positions per batch, 2T tokens total, ~10 epochs).

	SPT	SFT	PT
Loss-bearing positions	target tokens	target tokens	all non-padding
Pretraining metrics (MMLU, ARC-C)	↑	↓	↑
Reasoning metrics	↑	↑	≈
Multi-epoch overfitting	not observed	at 2–4 epochs	not observed
Token positions per batch	≥128M	128K–1M	8–32M

SPT improves both metric families with no loss cliff over 10 epochs (Figures 9–11). The explanation: each update aggregates supervision from over a thousand sequences and >100M token positions, suppressing the rapid specialization and memorization of small-batch SFT. This directly challenges the received view that SFT necessarily causes catastrophic forgetting.

RL: GSPO (length-normalized sequence-level importance ratios and sequence-level clipping) plus DAPO's clip-higher and dynamic sampling; math first, then code; Guru-RL data; staged 32K → 64K response lengths.

3.5 Thinking-model results (Tables 3–4)

Loopie-20B-A2B Thinking (3.5T pretraining tokens) against peers:

	Qwen3-30B-A3B Thinking (36T)	Nemotron 3 Nano 30B-A3B (25T)	Nemotron Cascade 2 (25T)	GPT-OSS 20B-A2B	**Loopie-20B-A2B (3.5T)**
MMLU	85.83	80.52	81.22	81.64	**81.28**
ARC-Challenge	96.67	91.96	93.86	92.42	93.52
BBH	86.59	68.76	75.86	84.76	82.28

	Qwen3-30B-A3B Thinking (36T)	Nemotron 3 Nano 30B-A3B (25T)	Nemotron Cascade 2 (25T)	GPT-OSS 20B-A2B	**Loopie-20B-A2B (3.5T)**
IFEval	85.58	77.05	79.21	70.03	84.72
HumanEval	96.95	86.59	79.88	92.07	89.02
AIME 24 (avg@8)	90.10	85.00	93.33	88.33	92.09
AIME 25	83.75	74.17	91.25	87.50	83.75
AMC	96.73	91.80	93.57	91.05	94.21

Against the two 25T Nemotron models trained on the same data source, Loopie beats both on 5 of 7 knowledge/general benchmarks with under 1/7 of the tokens.

Loopie-6B-A0.6B: MMLU 78.36, **AIME24 80.42, AIME25 70.83** (Ouro 2.6B Thinking: 62.50 / 51.67), MATH-500 93.80. An 80 on AIME24 from 0.6B active parameters is the most striking number in the table.

4. Why it matters

1. **The first compute-matched win.** This is the boundary between "academic toy" and "scalable pretraining paradigm". Earlier comparisons were either parameter-matched (favoring loops) or FLOP-matched (disfavoring them); Loopie introduces measured-wall-clock matching — the third standard and the one industry cares about.
2. **Loops meet MoE.** Huginn's §9 predicted that recurrence (reasoning) and MoE (storage) are complementary and "a future architecture would contain both". Loopie is the first large-scale confirmation.
3. **Layer-loop has independent corroboration.** Training-Free Looped Transformers finds, without any training, that MoE needs layer-mode; SMELT (2609.01343) arrives at "loop the middle half twice" under FLOP/parameter/KV triple matching. Three papers, three experimental paradigms, one conclusion.
4. **SPT may be a contribution independent of looping.** Nothing in it is specific to looped models; if it reproduces on dense models it changes default post-training practice.

5. Limitations and caveats

- **"Compute-matched" means matched measured step time; Loopie's nominal FLOPs are 42% higher.** Fair for training cost, not for inference cost: Loopie-20B-A2B executes 27×2=54 block passes per token versus 48 for the reference, with a larger looped KV cache. No inference-latency comparison is reported.
- The Qwen3-30B-A3B comparison is a **self-reproduction** trained for 800B tokens (the official model: 36T); the gap to official Qwen3 Thinking in Table 3 (MMLU 81 vs 86, BBH 82 vs 87) is real.
- The scaling ladder tops out at 1B active parameters; the 20B-A2B compute-matched comparison is a single pair.
- In Figure 6's $N\times$ loop vs. $N\times$ layer comparison, the 2× layer baseline used "substantially more" compute than the 2× loop model (the authors say so); that figure cannot be read as "2× looping is dominated by 2× layers".
- Pretraining code is not released; data is entirely public Nemotron, so reproducibility is reasonable.

6. Relation to other work

- Predecessors: UT (layer tying), ALBERT, MoEUT (the first MoE + UT combination, but only on par with standard transformers at small scale), Relaxed Recursive Transformers.
- Same group: LoopCoder-v2 (PLT, $R = 2$). The two papers reach $R = 2$ by compute matching and by mechanistic diagnosis respectively.
- Successors / concurrent: SMELT (2609.01343) fits Chinchilla-style scaling laws for compute-matched looped MoE, saving 6.8–18.0% training FLOPs and finding that the second visit reduces the attention sink; Schwethelm et al.'s $\varphi = 0.46$ explains why "trading parameters for loops" loses under FLOP matching — Loopie avoids that trade and buys width with the memory dividend instead.
- Contrast with Hidden Decoding (2607.08186, WeLM 617B): it argues looping is incompatible with pipeline parallelism and adds compute along the sequence dimension instead; Loopie's layer-loop exists precisely to make looping pipeline-friendly. These are the two competing routes to "more compute per token" today.

7. Practical takeaways

- Match compute by measured step time, not FLOPs, when comparing looped models: activation-memory savings are a real dividend of looped architectures and should count.
- Default to layer-loop on MoE backbones; it is also the easier choice for pipeline parallelism.
- Under frontier pretraining budgets use $R = 2$; leave larger R to adaptive inference-time compute.

- Try SPT on any model: treat SFT data as pretraining data (large batch, long sequences, many epochs) and watch whether pretraining metrics stop degrading.
- Inference cost of looped models remains open; measure latency and KV cache before deploying.

Appendix: quick numbers

- Loopie-20B-A2B: $D = 2304$, $L = 27$, $R = 2$, nominal compute 1.42×, activation memory 0.63×, step time equal to Qwen3-30B-A3B (MBS 2 vs 1).
- Compute-matched run: 800B tokens, overtakes after ~600B. Scaling ladder 4/4 wins.
- Pretraining 3.5T (2.28T = 4 epochs of HQ + 1.26T annealing); SPT 2T, 10 epochs; RL GSPO+DAPO, 32K→64K.
- 20B-A2B Thinking: MMLU 81.28, ARC-C 93.52, BBH 82.28, IFEval 84.72, AIME24 92.09, AMC 94.21.
- 6B-A0.6B Thinking: AIME24 80.42, AIME25 70.83, MATH-500 93.80.

Glossary

Term	One-line meaning
Recurrent depth / looped transformer	The same parameters are applied repeatedly along depth; effective depth = stored layers × loop count, parameters unchanged
model-loop / block-loop / layer-loop	Loop the whole stack / only a middle segment / each layer several times before the next; see `assets/figures/diagrams/loop_granularity.png`
Input injection	Re-feed the input embedding at every loop; without it the iteration is unstable
Truncated backprop	Compute gradients through only the last k loops so memory is independent of the loop count
Adaptive halting (ACT)	Let each token decide how many loops it needs
Path independence	Different random initial states converge to the same behavior
KV cache	Cached attention keys/values; looped models add one copy per loop, the main memory bottleneck
MoE	Mixture of experts: each token activates a small subset of parameters; “20B-A2B” = 20B total, 2B active
FLOP-matched vs measured-compute-matched	Aligning theoretical compute vs aligning measured training step time — the activation memory looping saves only shows up in the latter
Latent reasoning vs explicit CoT	Extra computation in hidden space vs intermediate steps written as text tokens
Fixed point	A state that another loop leaves unchanged; one line of work trains loops to converge to fixed points for stability
PLT / CLP	Parallel Loop Transformer / cross-loop position offset — shift the previous round’s states by one token so rounds run in parallel